Módulo 3

REVISÃO E APROFUNDAMENTO DE PANDAS E ANÁLISE DE DADOS COM PYTHON

SUMÁRIO

MÓDULO 3

REVISÃO E APROFUNDAMENTO DE PANDAS E ANÁLISE DE DADOS COM PYTHON

1	Como Vai Funcionar Esse Módulo?	3
2	ETL, o Desafio do Projeto e Carregando os Dados	4
3	Parâmetros do read_csv, Inspeção e Estrutura da Base	10
4	info(), describe() e Seleção de Colunas	18
5	Excluindo Colunas, Valores Vazios e Conversão de Data	24
6	Padronização de Texto, Duplicatas e Nova Coluna	31
7	Encadeamento, np.where() e .map()	36
8	isna(), .loc e .iloc	40
9	sort_values(), Filtro Condicional e to_csv()	43
10	Filtro por Data, Condições Múltiplas e groupby()	47
11	groupby() Avançado	50
12	Exibir Tabela Completa e merge()	54
13	Tipos de Join, Bateu Meta e Série Temporal	57
14	Revisão das Funções do Pandas	60
15	[Opcional] Aula de Orientação sobre o Projeto Exercício de Revisão	66
16	[Opcional] Projeto Exercício de Revisão — ETL, Engenharia de Colunas e Análises	67

Módulo 3 – Como Vai Funcionar Esse Módulo? (1 / 1)

Como Vai Funcionar Esse Módulo?

Esse módulo é dedicado a revisar e aprofundar tudo de **Pandas** que você precisa para trabalhar com dados em Python — e a proposta é fazer isso de um jeito bem diferente de uma lista solta de funções.

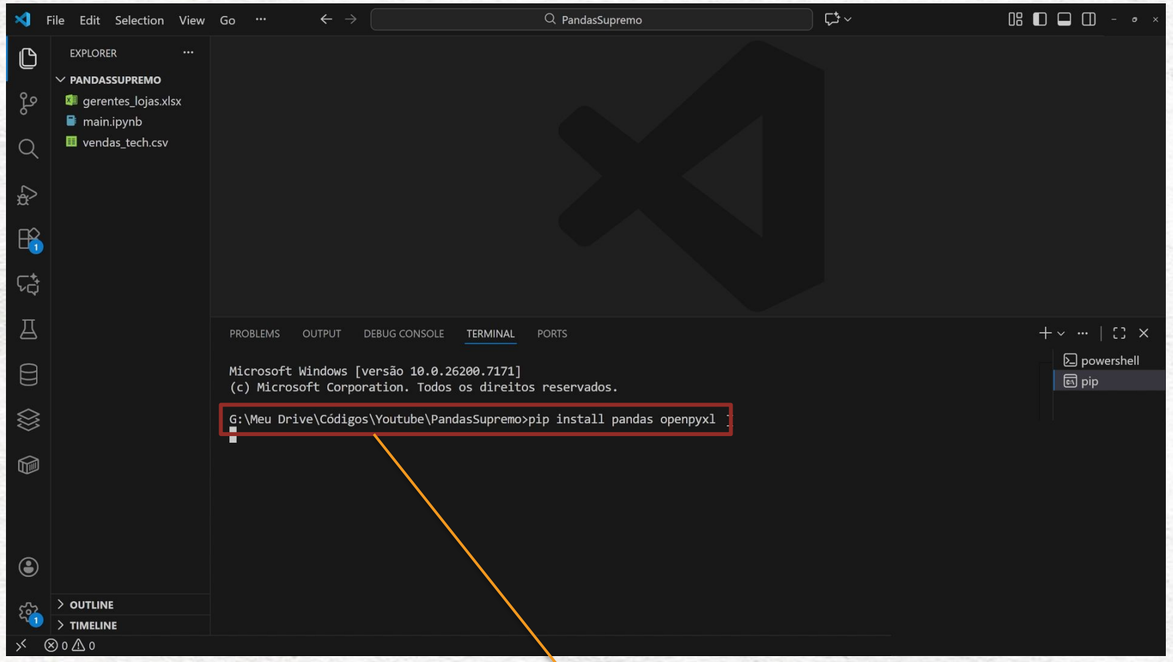
Da **Aula 2 até a Aula 13**, tudo vai ser ensinado dentro de **um único projeto real**, do início ao fim: uma base de vendas que precisa ser carregada, tratada, enriquecida com novas colunas e analisada para responder perguntas de negócio de verdade — faturamento por loja, ranking de produtos, gerentes que bateram a meta, tendência de vendas ao longo do tempo.

Você vai passar por **ETL** (extrair, transformar, carregar), **tratamento de dados** (valores vazios, tipos, duplicatas, padronização de texto), **engenharia de colunas**, **filtros**, **agrupamentos**, **junção de bases** e **análise de série temporal** — nessa ordem, porque é a ordem natural de qualquer projeto de dados no mercado.

Depois desse percurso principal, a **Aula 14** traz mais uma rodada de revisão, e as **Aulas 15 e 16** trazem um **Projeto Exercício de Revisão opcional**: o mesmo projeto, reconstruído do zero, para quem quiser praticar mais antes de seguir em frente. Nada ali é conteúdo novo — é só reforço para quem sentir que precisa.

Recomendação: acompanhe digitando o código junto, não só assistindo. Pandas se aprende praticando, e cada aula deste módulo foi pensada para você sair de um resultado real, não de um exemplo solto.

Módulo 3 – ETL, o Desafio do Projeto e Carregando os Dados (1 / 6)



Instala o Pandas e a engine do Excel de uma vez

Pandas: a Ferramenta de ETL do Python

O **Pandas** é a ferramenta mais usada na área de dados em Python. Essencialmente, ele cobre todo o processo de **ETL** — sigla para **Extract, Transform, Load**: extrair os dados, transformá-los (corrigir problemas da base) e carregá-los para análise.

Diferente de só carregar os dados para outro lugar, o Pandas permite fazer as análises **dentro dele mesmo**.

Ao lado: pip install pandas openpyxl — o Pandas não vem instalado por padrão no Python. O **openpyxl** é a engine que o Pandas usa por trás dos panos para ler arquivos Excel; cada formato de base pode exigir uma biblioteca adicional própria.

O projeto usa duas bases reais: **vendas_tech.csv** e **gerentes_lojas.xlsx** — CSV e Excel, os formatos mais comuns no mercado de trabalho.

Por que .ipynb em vez de .py?

Em vez de um arquivo Python comum (**.py**), o projeto usa um **notebook Jupyter** — extensão **.ipynb** (aqui, **pandas_supremo.ipynb**, nunca **pandas.ipynb** — esse nome bateria com o da própria biblioteca e causaria conflito na hora de importar).

Analistas, cientistas e engenheiros de dados costumam trabalhar com notebooks, principalmente no início e durante o desenvolvimento de um projeto — é o formato padrão do Jupyter e do Google Colab; no VS Code, basta ter a extensão do Jupyter instalada.

A diferença prática: um notebook é dividido em **células de código** (botão **+ Code**) que rodam de forma independente, uma a uma — ideal para visualizar dados no meio do processo, sem precisar rodar o script inteiro de novo a cada mudança.

```
# nome do notebook do projeto
pandas_supremo.ipynb

# evite: conflita com a biblioteca
# pandas.ipynb
```

O Desafio do Projeto: Duas Bases de Dados

Antes de sair digitando código, todo projeto de dados começa com um **desafio de negócio**: o gestor da empresa pediu um relatório de vendas por gerente — quem vende mais, qual produto cada gerente mais vende — e a tendência de faturamento mensal (subindo, caindo ou estável).

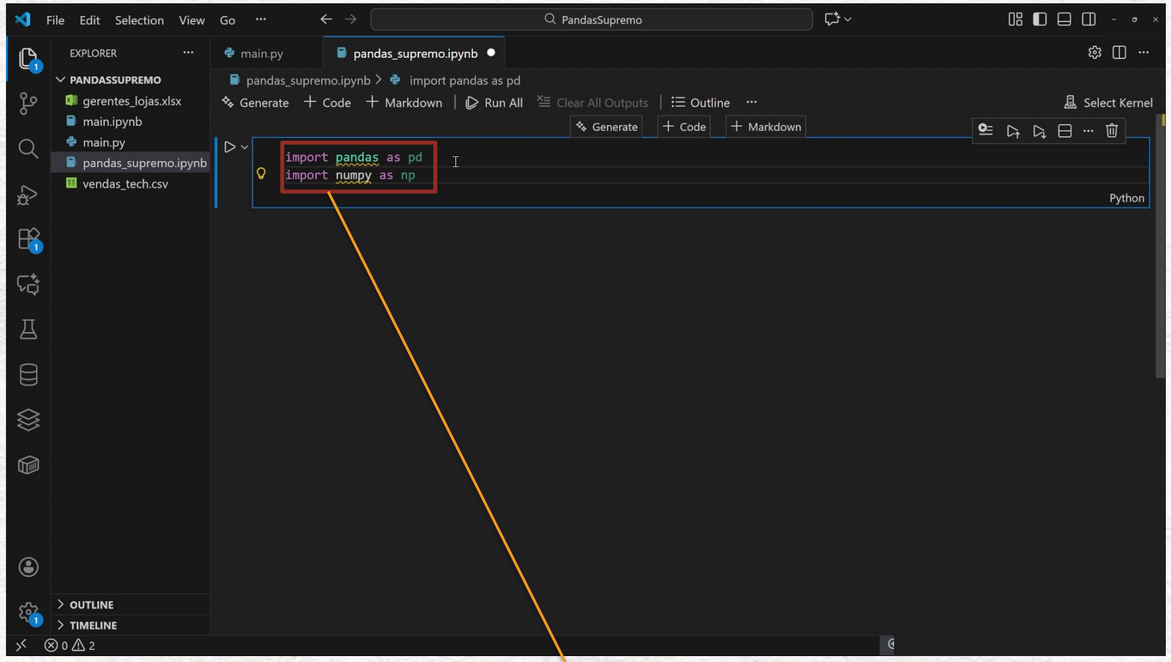
Para responder essas perguntas, o projeto usa **duas bases de dados**, cada uma num formato diferente — o mesmo cenário que você encontra no mercado de trabalho.

Ao lado: vendas_tech.csv (a base de vendas, em CSV) e **gerentes_lojas.xlsx** (a base de gerentes por loja, em Excel).

Ter as duas juntas no projeto força você a lidar com formatos diferentes — exatamente o tipo de situação que o Pandas resolve.

```
# as duas bases do projeto
vendas_tech.csv          # CSV
gerentes_lojas.xlsx      # Excel
```

Módulo 3 – ETL, o Desafio do Projeto e Carregando os Dados (4 / 6)



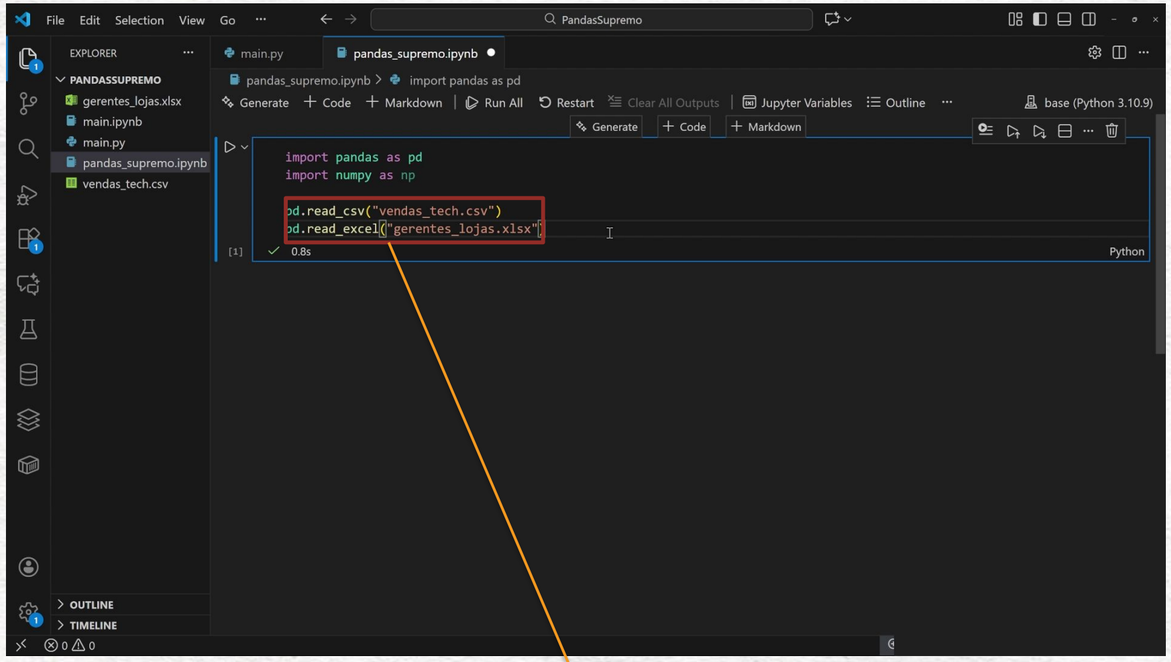
Pandas como pd, NumPy como np — padrão do mercado

Importando as Bibliotecas: Pandas e NumPy

Com o desafio definido, o primeiro passo do código é importar as bibliotecas. O **Pandas** é sempre importado com o apelido **pd** — convenção usada por praticamente todo mundo no mercado, só para escrever menos ao longo do caminho.

Junto dele, também vale importar o **NumPy** como **np**. O NumPy é quem, por trás dos panos, faz as operações do Pandas de forma eficiente — por isso os dois aparecem juntos no início de praticamente todo projeto de dados em Python.

Se o VS Code não reconhecer as bibliotecas, selecione a versão do Python no **Select Kernel** (canto superior direito) — e, num notebook novo, ele pode pedir para instalar o **ipykernel** antes de rodar a primeira célula. É só clicar em Instalar.



Carregando os Dados: read_csv e read_excel

Com as bibliotecas importadas, a primeira grande funcionalidade do Pandas entra em cena: **carregar as informações** da base de dados para dentro do código.

O comando começa sempre com **pd.read_**, seguido do formato do arquivo — o Pandas tem um **read_** para praticamente qualquer formato: clipboard, CSV, Excel, HTML, JSON, Parquet, SQL e outros.

Ao lado: pd.read_csv("vendas_tech.csv") para a base em CSV e **pd.read_excel("gerentes_lojas.xlsx")** para a base em Excel — cada formato usa o método correspondente.

É exatamente por isso que os formatos diferentes das duas bases do projeto importam: eles determinam qual comando de leitura você vai usar.

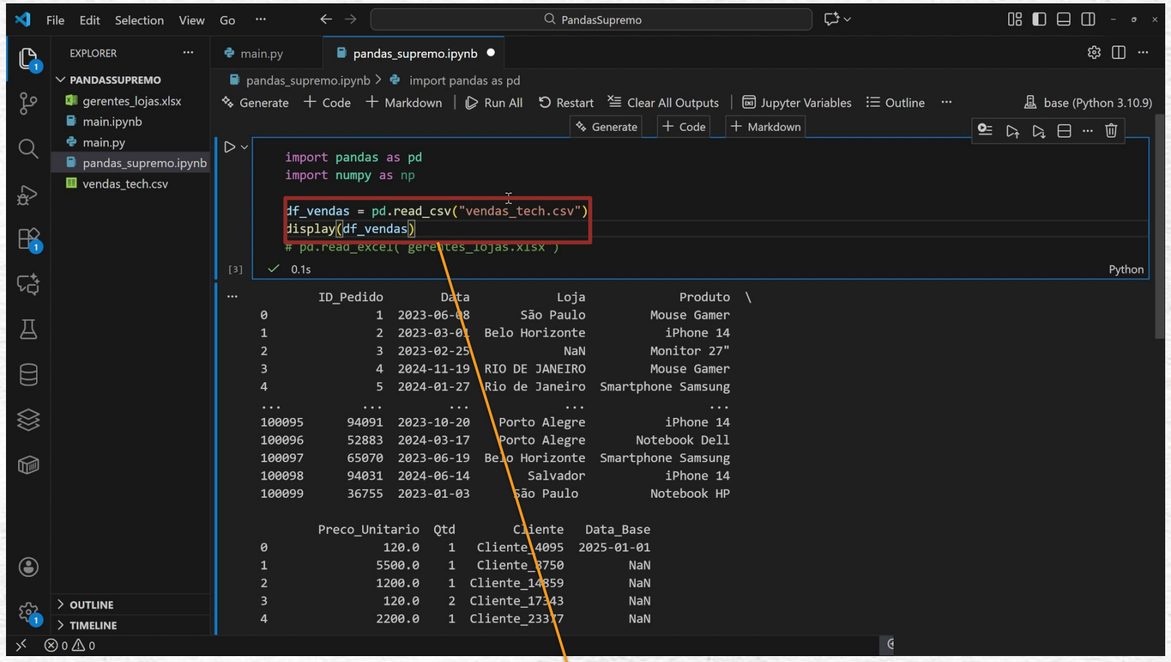
Módulo 3 – ETL, o Desafio do Projeto e Carregando os Dados (6 / 6)

DataFrame: a Tabela do Pandas

Quando o Pandas lê uma base de dados, ele precisa guardar essa tabela em algum lugar dentro do Python. Esse lugar tem nome: **DataFrame** — a estrutura básica do Pandas, a tabela propriamente dita.

Boa prática: em vez de chamar a variável de **dataframe** por extenso, o mercado usa o prefixo **df_** seguido de um nome intuitivo — aqui, **df_vendas**, porque é a base de vendas. Só de ler o nome já dá para saber o que a variável guarda.

Nos arquivos **.ipynb**, no lugar do **print()** existe o **display()** — mostra a mesma tabela, só que de forma bem mais visual e organizada.



Nome intuitivo com df_ e display() no lugar do print()

Módulo 3 – Parâmetros do read_csv, Inspeção e Estrutura da Base (1 / 8)

encoding: Lidando com Caracteres Especiais

O pd.read_csv aceita vários parâmetros opcionais para lidar com bases de dados que fogem do padrão — e no Brasil isso acontece o tempo todo, principalmente com acento, cedilha e til. Dica: digite uma vírgula após o nome do arquivo dentro do read_csv(...) para o autocomplete listar todos os parâmetros disponíveis.

O primeiro parâmetro é o encoding: define como os caracteres especiais são interpretados na hora da leitura. Se a base tiver algo como "São Paulo" e der erro ao carregar, o problema costuma ser esse.

Por padrão o Pandas já usa encoding="utf-8". Quando dá erro, o mais comum é resolver com encoding="latin-1", como ao lado — o mais usado para bases com caracteres em português. Na dúvida, jogue a mensagem de erro numa IA.

```
pd.read_csv(
    "vendas_tech.csv",
    encoding="latin-1",
)
```

encoding="latin-1" resolve acentos em português

Módulo 3 – Parâmetros do read_csv, Inspeção e Estrutura da Base (2 / 8)



sep: o Separador de Colunas

O segundo parâmetro do dia é o **sep**, o separador de colunas do arquivo. O Pandas espera vírgula por padrão — é assim que ele sabe onde termina uma coluna e começa a próxima.

O problema é que uma planilha exportada do Excel em português costuma usar **ponto e vírgula** como separador em vez de vírgula. Se você não avisar o Pandas disso, ele vai tentar separar pela vírgula e as colunas vêm todas coladas numa só.

Nesse caso, é só passar **sep=";"** na hora de ler o arquivo, como ao lado — o Pandas passa a usar o ponto e vírgula como referência para separar cada coluna corretamente.

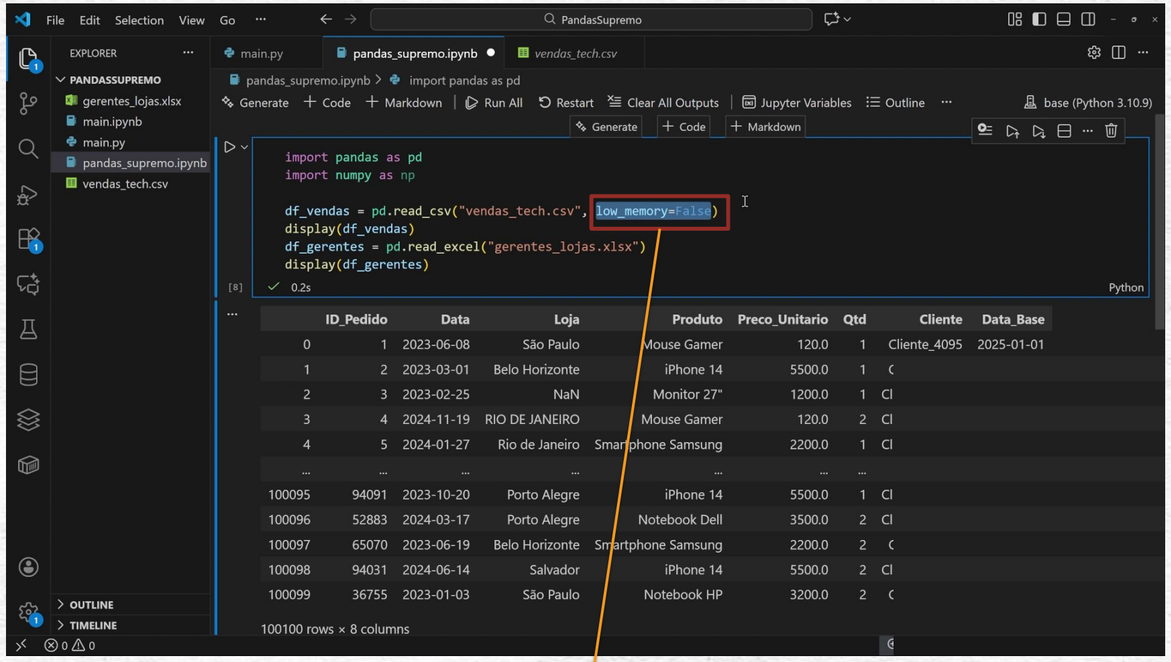
Módulo 3 – Parâmetros do read_csv, Inspeção e Estrutura da Base (3 / 8)

low_memory=False: o Aviso de Colunas Mistas

Terceiro parâmetro importante: **low_memory=False**. Ele aparece para resolver um aviso comum — não é erro, é só aviso — de que uma coluna tem **tipos de dados misturados**, como datas e textos juntos na mesma coluna.

Por padrão, o Pandas lê a base em pedaços para economizar memória, e por isso pode não identificar corretamente o tipo de cada coluna de uma vez. Passando **low_memory=False**, você diz para ele carregar a base inteira de uma só vez — o aviso desaparece, como no resultado ao lado.

Detalhe importante: tudo isso funciona igual em um arquivo **.py** comum, exceto o **display()**, que só existe nos notebooks — fora deles, use **print()** no lugar.



Módulo 3 – Parâmetros do read_csv, Inspeção e Estrutura da Base (4 / 8)

	ID_Pedido	Data	Loja	Produto	P
0	1	2023-06-08	São Paulo	Mouse Gamer	
1	2	2023-03-01	Belo Horizonte	iPhone 14	
2	3	2023-02-25	NaN	Monitor 27"	
3	4	2024-11-19	RIO DE JANEIRO	Mouse Gamer	
4	5	2024-01-27	Rio de Janeiro	Smartphone Samsung	

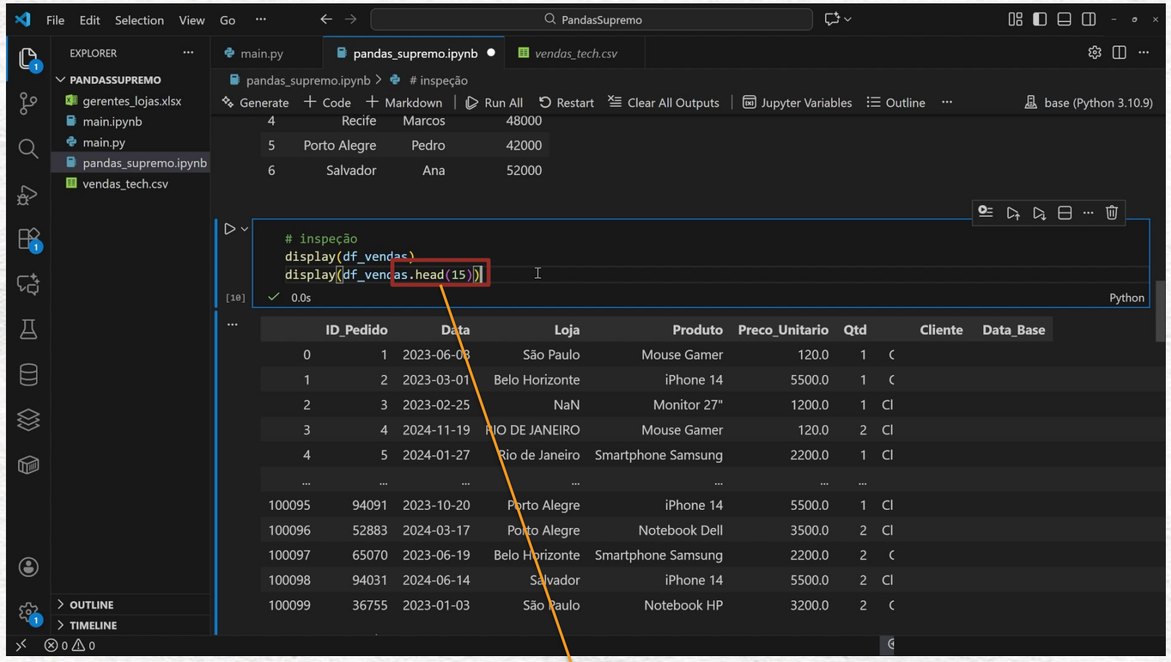
O Que Procurar: Problemas Comuns na Base

Depois de carregar e visualizar a base, começa a etapa de **inspeção**: olhar com atenção para encontrar problemas antes de sair analisando os dados.

Ao lado, um pedaço real da base de vendas já mostra dois problemas clássicos: a linha 2 tem a loja vazia (**NaN**, "not a number" — o Pandas usa esse marcador para qualquer valor ausente, não só números), e as linhas 3 e 4 têm a mesma cidade escrita de duas formas diferentes — **RIO DE JANEIRO** e **Rio de Janeiro**.

Para o Pandas, essas duas strings são valores completamente diferentes — o que vai bagunçar qualquer contagem por loja mais adiante. Encontrar esse tipo de problema é o objetivo da inspeção, e as ferramentas a seguir ajudam a fazer isso rapidamente.

Módulo 3 – Parâmetros do read_csv, Inspeção e Estrutura da Base (5 / 8)



head() e tail(): as Pontas da Base

Por padrão, **display(df_vendas)** mostra só as 5 primeiras e as 5 últimas linhas da base — pouco para inspecionar algo do tamanho da nossa.

Para ver mais que isso, existe o **head()**, que traz o início da base — e o **tail()**, o final dela. Os dois aceitam a quantidade de linhas como argumento, como ao lado.

df_vendas.head(15) traz as 15 primeiras linhas, **df_vendas.tail(15)** as 15 últimas — dá bem mais flexibilidade do que o padrão de 5 linhas do display, e ajuda a pegar problemas que não aparecem logo nas primeiras linhas.

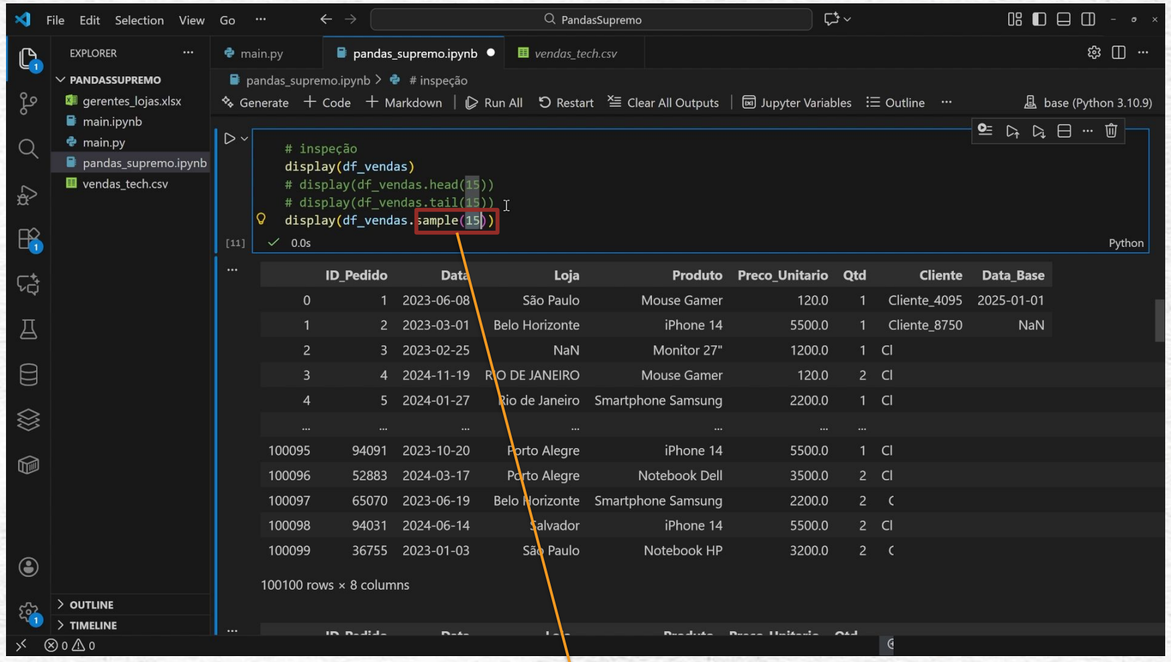
Módulo 3 – Parâmetros do read_csv, Inspeção e Estrutura da Base (6 / 8)

sample(): uma Amostra Aleatória da Base

Diferente do **head()** e do **tail()**, que sempre mostram as pontas da base, o **sample()** pega linhas **aleatórias** — nem as primeiras, nem as últimas.

A sintaxe é a mesma: **df_vendas.sample(15)** traz 15 linhas escolhidas ao acaso. É útil quando você quer ter uma ideia geral da base sem viés de posição — as primeiras ou últimas linhas podem não representar bem os dados como um todo.

Com **head()**, **tail()** e **sample()** na caixa de ferramentas, você já consegue explorar qualquer base de dados de vários ângulos antes de começar a tratá-la.



Módulo 3 – Parâmetros do read_csv, Inspeção e Estrutura da Base (7 / 8)

```
(100100, 8)

['ID_Pedido',
 'Data',
 'Loja',
 'Produto',
 'Preco_Unitario',
 'Qtd',
 'Cliente',
 'Data_Base']
```

shape e columns: Tamanho e Nomes das Colunas

Toda base de dados do Pandas é organizada em **colunas** (com um rótulo, o nome da coluna) e **linhas**. Duas ferramentas rápidas ajudam a entender o tamanho e a composição da base sem precisar rodar um **display()** completo.

Ao lado: df_vendas.shape — repare que não tem parênteses, porque **shape** não é uma função, é uma característica da base. Ele devolve uma dupla com (quantidade de linhas, quantidade de colunas) — aqui, 100.100 linhas e 8 colunas.

Já **list(df_vendas.columns)** devolve todos os nomes das colunas como uma lista Python comum — dá para tratar, percorrer ou filtrar essa lista como qualquer outra.

O Índice: os Rótulos das Linhas

Assim como as colunas têm um rótulo (o nome), as linhas também têm — o Pandas chama isso de **índice**.

Quando você carrega uma base do zero, o índice é criado automaticamente começando em **0** e indo até a última linha — no nosso caso, de 0 a 100.099, um **RangeIndex**.

Isso pode mudar: mais adiante, ao manipular a base, você vai poder definir uma coluna (como a loja) como índice — o que muda como você seleciona e cruza informações. Por enquanto, o importante é saber que o índice existe e, por padrão, começa em zero.

```
df_vendas.index
```

```
RangeIndex(start=0, stop=100100, step=1)
```

Módulo 3 – info(), describe() e Seleção de Colunas (1 / 6)

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100100 entries, 0 to 100099
Data columns (total 8 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   ID_Pedido       100100 non-null  int64
 1   Data            100100 non-null  object
 2   Loja            98099 non-null   object
 3   Produto         100100 non-null  object
 4   Preco_Unitario  100100 non-null  float64
 5   Qtd             100100 non-null  int64
 6   Cliente         100100 non-null  object
 7   Data_Base       1 non-null       object
dtypes: float64(1), int64(2), object(5)
memory usage: 6.1+ MB
```

info(): a Estrutura da Base num Golpe Só

Depois de explorar linhas soltas com **head()**, **tail()** e **sample()**, chega a hora de ver a base inteira de uma vez. **display(df_vendas.info())** traz um resumo técnico completo.

Ele mostra, para cada coluna: o nome, o **Non-Null Count** (quantos valores estão preenchidos) e o **Dtype** (o tipo de dado — **int64** para inteiro, **float64** para número com casa decimal, **object** para texto).

No rodapé, um resumo geral: quantas colunas de cada tipo a base tem e quanta memória ela ocupa. É o primeiro lugar para olhar antes de decidir o que precisa de tratamento.

Módulo 3 – info(), describe() e Seleção de Colunas (2 / 6)

O Que o info() Revela: Tipos Errados e Coluna Vazia

Comparar o **Non-Null Count** de cada coluna com o total de linhas (100.100) já denuncia valores ausentes — a coluna **Loja** tem só 98.099 preenchidas, ou seja, quase 2 mil lojas em branco.

Destacado ao lado: a coluna **Data** está com **Dtype object** — ou seja, o Pandas está tratando as datas como texto puro, não como data de verdade. Isso vai impedir cálculos e ordenações que dependem de data mais adiante.

E a coluna **Data_Base** tem só **1 non-null** — de 100.100 linhas, só uma tem valor preenchido. Uma coluna praticamente vazia como essa costuma ser candidata a ser simplesmente descartada.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100100 entries, 0 to 100099
Data columns (total 8 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   ID_Pedido       100100 non-null  int64
 1   Data            100100 non-null  object
 2   Loja            98099 non-null   object
 3   Produto         100100 non-null  object
 4   Preco_Unitario  100100 non-null  float64
 5   Qtd             100100 non-null  int64
 6   Cliente         100100 non-null  object
 7   Data_Base       1 non-null       object
dtypes: float64(1), int64(2), object(5)
memory usage: 6.1+ MB
```

Módulo 3 – info(), describe() e Seleção de Colunas (3 / 6)

	ID_Pedido	Preco_Unitario	Qtd
count	100100.000000	100100.000000	100100.000000
mean	50004.810180	2000.204595	1.499101
std	28866.872543	1841.050733	1.241605
min	1.000000	40.000000	1.000000
25%	25008.750000	120.000000	1.000000
50%	50004.500000	1200.000000	1.000000
75%	75002.250000	3200.000000	1.000000
max	100000.000000	5500.000000	10.000000

describe(): Resumo Estatístico das Colunas Numéricas

Enquanto **info()** fala de estrutura, **display(df_vendas.describe())** fala de números: um resumo estatístico automático de toda coluna numérica (**int64** ou **float64**) da base — aqui, **ID_Pedido**, **Preco_Unitario** e **Qtd**.

Para cada uma, ele calcula: **count** (quantos valores), **mean** (a média) e **std** (o desvio padrão, o quanto os valores variam em torno da média). Depois vêm **min** e **max**, e os três quartis: **25%**, **50%** (a mediana — o valor bem no meio da base) e **75%**.

Sem entrar na estatística por trás, o describe já ajuda a enxergar rapidamente a faixa de valores da base — por exemplo, o **Preco_Unitario** varia de 40 a 5.500, com metade dos pedidos abaixo de 1.200.

Módulo 3 – info(), describe() e Seleção de Colunas (4 / 6)

```
display(df_vendas["Loja"])
```

```
0                São Paulo
1           Belo Horizonte
2                      NaN
3           RIO DE JANEIRO
4           Rio de Janeiro
                    ...
100095        Porto Alegre
100096        Porto Alegre
100097      Belo Horizonte
100098           Salvador
100099          São Paulo
Name: Loja, Length: 100100, dtype: object
```

Selecionando uma Coluna: colchete único

Com a base carregada e inspecionada, começa o **tratamento de dados** — e o primeiro passo é entender como selecionar colunas.

Para selecionar uma única coluna, você usa a base seguida do nome da coluna entre colchetes: **df_vendas["Loja"]**. O nome precisa ser **exatamente igual** ao que aparece na base — maiúsculas, minúsculas e espaços contam. Por isso vale a pena conferir a lista de colunas antes.

Repare no resultado ao lado: ele não vem formatado como tabela, só uma lista de valores com o índice do lado — isso porque selecionar uma única coluna devolve uma **Series**, não um DataFrame. O rodapé **Name: Loja, dtype: object** confirma isso.

Módulo 3 – info(), describe() e Seleção de Colunas (5 / 6)

Selecionando Várias Colunas: colchete duplo

Para selecionar mais de uma coluna, um colchete não basta — você precisa de **dois**: `df_vendas[["Loja", "Cliente"]]`.

O colchete de fora diz "quero selecionar colunas"; o de dentro é porque você está passando uma **lista** de nomes — por isso dois colchetes, não importa se são 2, 3 ou 10 colunas. Ao lado, a lista foi separada numa variável (**lista_colunas**) só para deixar mais claro, mas dá para escrever direto também.

Repare que agora o resultado vem formatado como tabela, com cabeçalho — porque selecionar mais de uma coluna sempre devolve um **DataFrame**, não uma Series. É basicamente assim que você cria uma nova base a partir de um pedaço de outra, mantendo os mesmos índices originais.

```
lista_colunas = ["Loja", "Cliente"]
display(df_vendas[lista_colunas])
```

	Loja	Cliente
0	São Paulo	Cliente_4095
1	Belo Horizonte	Cliente_8750
2	NaN	Cliente_14859
3	RIO DE JANEIRO	Cliente_17343

Módulo 3 – info(), describe() e Seleção de Colunas (6 / 6)

```
df_vendas["Preco_Unitario"] * df_vendas["Qtd"]

0     120.0
1    5500.0
2    1200.0
...
Name: Preco_Unitario, dtype: float64
```

Por que o Pandas é tão Rápido: Operações Vetorizadas

Uma **Series** funciona, por trás dos panos, como um array do **NumPy** — a biblioteca que a gente importou junto com o Pandas logo no início. É o NumPy quem faz as operações matriciais do Pandas de forma muito eficiente.

Na prática, isso significa que você pode multiplicar, somar ou comparar colunas inteiras de uma vez, como ao lado — sem escrever nenhum **for** percorrendo linha por linha.

Isso é possível porque Pandas e NumPy foram construídos em **C**, uma linguagem que permite gerenciar memória e executar operações de forma muito mais eficiente do que um loop em Python puro. É exatamente isso que torna essas bibliotecas o padrão do mercado de dados.

Módulo 3 – Excluindo Colunas, Valores Vazios e Conversão de Data (1 / 7)

Duas Formas de Excluir Colunas

Lembra da coluna **Data_Base**, que o **info()** mostrou com só 1 valor preenchido em 100.100 linhas? Chegou a hora de descartá-la.

Existem duas formas de excluir colunas. A primeira é selecionar só as colunas que você **quer manter** — tudo que não estiver na lista é descartado.

A segunda é o método **drop()**, passando **columns=** com a lista do que você **não quer**. Para excluir uma coluna só, é sempre mais direto do que reescrever a lista inteira das que você quer manter — e para excluir duas ou mais, basta colocar todas na mesma lista.

```python
# opcao 1: manter so o que voce quer
df_vendas = df_vendas[["Loja", "Cliente"]]

# opcao 2: excluir so o que voce nao quer
df_vendas = df_vendas.drop(columns=["Data_Base"])
```

```
# nao mexe na df_vendas original
df_analise = df_vendas.drop(columns=["Data_Base"])

display(df_analise)
```

```
100100 rows x 7 columns
```

Trabalhando numa Cópia: df_analise

Reparou que os exemplos anteriores reescreviam **df_vendas** por cima dela mesma? Isso edita a base original — e se você precisar rodar aquela célula de novo, ela já não vai ter mais a coluna que você tirou, o que pode gerar erro.

Por isso, a prática mais segura é criar uma **cópia** para fazer todos os tratamentos: **df_analise**. A base original **df_vendas** fica intacta, e você sempre pode voltar a ela se precisar recomeçar o tratamento do zero.

A única exceção é quando a base é tão grande que já está pesando na memória do computador — aí, criar uma cópia pode não ser viável. Fora esse caso, vale sempre criar a cópia.

Módulo 3 – Excluindo Colunas, Valores Vazios e Conversão de Data (3 / 7)

```python
# tratamento de dados
display(df_vendas["Loja"])
display(df_vendas[["Loja", "Cliente"]])

# colunas
df_analise = df_vendas.drop(columns=["Data_Base"])

# nulos
df_analise = df_analise.dropna()

# tipos de dados

# duplicatas

display(df_analise)
```

dropna(): Removendo Linhas com Valores Vazios

Para saber quais colunas têm valores vazios, basta olhar o **info()** que a gente já rodou — ele mostra o **Non-Null Count** de cada coluna. Como a **Data_Base** já foi excluída, sobrou só a coluna **Loja** com valores vazios.

A forma mais simples de tratar isso é o **dropna()**: ele varre a base inteira e remove **toda linha** que tenha pelo menos um valor vazio em qualquer coluna — porque um valor vazio pode contaminar qualquer análise que dependa dele.

Ao lado, o roteiro completo do tratamento até aqui: excluir coluna, tratar vazios, tipos de dados, duplicatas. Rodando o **dropna()**, a base cai de **100.100** para **98.099** linhas — exatamente as linhas onde a Loja estava vazia.

Módulo 3 – Excluindo Colunas, Valores Vazios e Conversão de Data (4 / 7)

fillna(): Preenchendo em vez de Excluir

Excluir 2 mil linhas só porque a Loja estava vazia pode jogar fora informação boa. Se você souber **por que** ela está vazia, dá para preencher em vez de excluir.

Aqui, a regra de negócio é: quando a Loja não está preenchida, foi uma **venda online** — informação que você descobre conversando com quem alimenta o sistema, ou observando o padrão dos dados.

Para preencher só essa coluna (não a base inteira), usa-se **fillna()** direto na coluna: **df_analise["Loja"].fillna("Online")**. Depois de rodar, o **info()** confirma: a coluna Loja passa a ter as 100.100 linhas preenchidas, sem precisar excluir nada.

```
df_analise["Loja"] = df_analise["Loja"].fillna("Online")

info() antes:  Loja   98099 non-null
info() depois: Loja  100100 non-null
```

Módulo 3 – Excluindo Colunas, Valores Vazios e Conversão de Data (5 / 7)

```
# tratamento de dados
display(df_vendas["Loja"])
display(df_vendas[["Loja", "Cliente"]])

# colunas
df_analise = df_vendas.drop(columns=["Data_Base"])

# nulos
# df_analise = df_analise.dropna() # excluir todas as linhas que têm 1 valor vazio
df_analise["Loja"] = df_analise["Loja"].fillna("Online")

# tipos de dados
df_analise["Data"] = pd.to_datetime(df_analise["Data"])

# duplicatas

display(df_analise)
display(df_analise.info())
```

Convertendo para Data: pd.to_datetime()

Vazios tratados, é hora do próximo problema que o **info()** revelou lá atrás: a coluna **Data** está com **Dtype object** — o Pandas está lendo as datas como texto puro, não como data de verdade.

Para converter, existe a função **pd.to_datetime()**, que transforma uma coluna para o formato **DateTime** — o padrão de datas do Python. Você pode **sobrescrever a própria coluna** (como ao lado) ou salvar numa **coluna nova**, mantendo a original intacta.

Ao lado, o roteiro até aqui: exclusão de coluna, vazios tratados (o **dropna()** ficou comentado — no final só se usou **fillna()**) e agora a conversão com **to_datetime()**. Valor que não converte vira vazio, e precisa ser tratado de novo.

Módulo 3 – Excluindo Colunas, Valores Vazios e Conversão de Data (6 / 7)

O Parâmetro format: Códigos de Data

Para o Pandas entender corretamente uma data, muitas vezes é preciso dizer a ele qual é o formato dela — isso é o parâmetro **format**.

O formato é escrito com códigos: **%Y** é o ano com 4 dígitos, **%m** o mês com 2 dígitos, **%d** o dia com 2 dígitos. Se a data está no formato ano-mês-dia (**2024-01-27**), o parâmetro é **format="%Y-%m-%d"**.

Você não precisa decorar os códigos — a documentação oficial do Pandas (procure "pd to_datetime") tem a tabela completa, incluindo hora, minuto, segundo e dia da semana. Ao lado, os quatro mais usados no dia a dia.

```
df_analise["Data"] = pd.to_datetime(
    df_analise["Data"],
    format="%Y-%m-%d"
)
```

%Y	ano (4 digitos)	%m	mes (2 digitos)
%d	dia (2 digitos)	%H	hora (24h)

Módulo 3 – Excluindo Colunas, Valores Vazios e Conversão de Data (7 / 7)

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100100 entries, 0 to 100099
Data columns (total 7 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   ID_Pedido       100100 non-null  int64
 1   Data            100100 non-null  datetime64[ns]
 2   Loja            100100 non-null  object
 3   Produto         100100 non-null  object
 4   Preco_Unitario  100100 non-null  float64
 5   Qtd             100100 non-null  int64
 6   Cliente         100100 non-null  object
dtypes: datetime64[ns](1), float64(1), int64(2), object(3)
memory usage: 5.3+ MB
```

Conferindo o Resultado: datetime64[ns]

Depois de qualquer tratamento, vale a pena rodar o **info()** de novo para conferir se ele realmente resolveu o problema — é assim que você fecha o ciclo de cada tratamento.

Ao lado, o resultado depois da conversão: a coluna **Data** agora aparece como **datetime64[ns]** em vez de **object** — e continua com as 100.100 linhas preenchidas, sinal de que nenhuma data ficou vazia na conversão.

Sendo uma data de verdade agora, dá para comparar datas, ordenar por data, calcular uma série histórica — nada disso funcionava enquanto ela era só texto.

Módulo 3 – Padronização de Texto, Duplicatas e Nova Coluna (1 / 5)

```python
# tipos de dados
df_analise["Data"] = pd.to_datetime(df_analise["Data"], format="%Y-%m-%d")

# padronizacao
df_analise["Loja"] = df_analise["Loja"].str.strip()
df_analise["Loja"] = df_analise["Loja"].str.title()
```

Padronização de Texto: por que ela importa

Um problema que só aparece quando você já está analisando: "Rio de Janeiro" e "RIO DE JANEIRO" são a mesma cidade para uma pessoa, mas para o Pandas são **dois valores diferentes** — o que separa as vendas da mesma loja em duas linhas do relatório.

Para trabalhar com texto de forma eficiente no Pandas (sem for), usa-se **.str**, que disponibiliza as funções de texto do Python direto na coluna. Existem dezenas delas — aqui, duas bastam para resolver os dois problemas clássicos de padronização.

strip() remove espaços em branco no início e no fim do texto (padronização de espaços). **title()** deixa a primeira letra de cada palavra maiúscula e o resto minúsculo (padronização de capitalização) — dá para usar **lower()** no lugar, se preferir tudo minúsculo.

```
df_analise["Loja"] = df_analise["Loja"].str.strip().str.title()
```

```
antes:   "RIO DE JANEIRO", " rio de janeiro "
depois:  "Rio De Janeiro", "Rio De Janeiro"
```

Antes e Depois: Nomes de Loja Padronizados

Encadeando os dois métodos numa linha só, **strip()** tira os espaços indesejados e **title()** arruma a capitalização — os dois de uma vez.

Ao lado, o efeito prático: **"RIO DE JANEIRO"** e **" rio de janeiro "** (com espaços sobrando) viram os dois **"Rio De Janeiro"** — agora, sim, o Pandas reconhece como o mesmo valor.

Um cuidado: essa mudança fica só na **df_analise**, a cópia que você está tratando — a **df_vendas** original continua com os dados crus, do jeito que foram carregados.

Módulo 3 – Padronização de Texto, Duplicatas e Nova Coluna (3 / 5)

drop_duplicates(): Removendo Linhas Idênticas

Último tratamento da base: linhas **completamente duplicadas** — dois pedidos com todas as informações iguais, o que não faz sentido numa tabela de pedidos e costuma indicar um erro no preenchimento.

O método **drop_duplicates()** resolve isso direto: **df_analise = df_analise.drop_duplicates()**.

Ao lado, o resultado: a base cai de **100.100** para **100.000** linhas — as 100 duplicatas foram removidas. Rodando o **info()** de novo, tudo confirma certinho. Assim termina a etapa de tratamento de dados.

```
    df_analise["Loja"] = df_analise["Loja"].str.strip()
    df_analise["Loja"] = df_analise["Loja"].str.title()

    # duplicatas
    df_analise = df_analise.drop_duplicates()
```

```
100000 rows × 7 columns
```

```
<class 'pandas.core.frame.DataFrame'>
Index: 100000 entries, 0 to 99999
Data columns (total 7 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   ID_Pedido       100000 non-null  int64
```

```python
df_analise = df_analise.drop_duplicates(
    subset=["ID_Pedido"],
    keep="first"
)
```

subset e keep: Controlando o Que Conta como Duplicata

E se duas linhas tiverem o mesmo **ID_Pedido**, mas datas diferentes — porque duas pessoas cadastraram o mesmo pedido em dias diferentes? Ainda é o mesmo pedido, então ainda deveria ser removido — mas nem todas as colunas estão idênticas.

Para isso existe o parâmetro **subset**: uma lista de colunas que define o que conta como duplicata. Passando só **["ID_Pedido"]**, ele ignora as outras colunas e remove qualquer linha com o mesmo ID — dá para combinar mais de uma coluna também (ex.: ID + Loja).

O outro parâmetro é **keep**: qual cópia manter quando acha uma duplicata. Por padrão já é **"first"** (mantém a primeira, remove as seguintes) — dá para passar **"last"** para manter a última em vez disso.

Módulo 3 – Padronização de Texto, Duplicatas e Nova Coluna (5 / 5)

```
# faturamento
df_analise["Faturamento"] = df_analise["Qtd"] * df_analise["Preco_Unitario"]

# regiao

# forma de venda

display(df_analise)
```

✓ 0.0s

	ID_Pedido	Data	Loja	Produto	Preco_Unitario	Qtd	Cliente	Faturamento
0	1	2023-06-08	São Paulo	Mouse Gamer	120.0	1	Cliente_4095	120.0
1	2	2023-03-01	Belo Horizonte	iPhone 14	5500.0	1	Cliente_8750	5500.0
2	3	2023-02-25	Online	Monitor 27"	1200.0	1	Cliente_14859	1200.0
3	4	2024-11-19	Rio De Janeiro	Mouse Gamer	120.0	2	Cliente_17343	240.0
4	5	2024-01-27	Rio De Janeiro	Smartphone Samsung	2200.0	1	Cliente_23377	2200.0

Engenharia de Colunas: Criando a Coluna de Faturamento

Com o tratamento concluído, começa a etapa de análise — e, às vezes, a análise pede uma coluna que ainda não existe. Criar essas colunas novas é o que se chama **engenharia de colunas** (feature engineering).

A primeira e mais direta forma de criar uma coluna: uma **operação entre colunas existentes**. Para calcular o faturamento por pedido, basta multiplicar preço unitário pela quantidade: **df_analise["Faturamento"] = df_analise["Qtd"] * df_analise["Preco_Unitario"]**.

Repare a importância do tratamento feito antes: se **Qtd** tivesse algum valor vazio, o cálculo do faturamento também sairia vazio naquela linha. Tudo que você constrói daqui para frente depende da base já estar tratada corretamente.

Módulo 3 – Encadeamento, np.where() e .map() (1 / 4)

```python
# padronizacao
df_analise["Loja"] = df_analise["Loja"].str.strip()
df_analise["Loja"] = df_analise["Loja"].str.title()

# encadeamento
df_analise["Loja"] = df_analise["Loja"].fillna("Online").str.strip().str.title()
```

Encadeamento: Várias Operações numa Linha Só

As três operações da coluna **Loja** (**fillna**, **strip**, **title**) não precisam ficar em linhas separadas — dá para **encadear** tudo numa linha só, como destacado ao lado: **fillna("Online").str.strip().str.title()**.

Funciona porque cada operação devolve uma coluna como resposta, e essa coluna vira o ponto de partida da próxima — o resultado do **fillna()** recebe o **strip()**, e o resultado desse recebe o **title()**.

Recomendação: comece sempre escrevendo em linhas separadas — assim dá para visualizar o resultado de cada etapa — e só depois, se quiser, junte tudo numa linha encadeada.

Módulo 3 – Encadeamento, np.where() e .map() (2 / 4)

np.where(): Criando Colunas Condicionais

Com a coluna de faturamento pronta, a segunda forma de criar colunas é aplicar uma **condição**. Aqui, o objetivo é uma coluna que diga se a venda foi **online** ou **presencial**.

Para colunas condicionais, o recomendado é o **np.where()** — funciona como um **if** aplicado direto na coluna inteira: você passa a condição, o valor se for verdadeira e o valor se for falsa.

Ao lado: se **Loja** for igual a **"Online"**, preenche **"Online"**; caso contrário, **"Presencial"**. Dá para encadear vários **np.where()** um dentro do outro se precisar de mais de duas opções.

```
df_analise["Forma_de_Venda"] = np.where(
    df_analise["Loja"] == "Online",
    "Online",
    "Presencial"
)
```

Loja	Forma_de_Venda
Sao Paulo	Presencial
Online	Online
Curitiba	Presencial

Módulo 3 – Encadeamento, np.where() e .map() (3 / 4)

```
# regiao
display(df_analise["Loja"].unique())



display(df_analise)
```

✓ 0.0s

```
array(['São Paulo', 'Belo Horizonte', 'Online', 'Rio De Janeiro',
       'Salvador', 'Recife', 'Curitiba', 'Porto Alegre'], dtype=object)
```

Descobrindo os Valores Únicos: .unique()

Última coluna nova: **região** — sudeste, nordeste, sul, ou online. Antes de criar essa coluna, é preciso saber quais cidades existem na base.

Para isso existe o **.unique()**: aplicado numa coluna, devolve todos os valores diferentes que aparecem nela, sem repetição — ao lado, **df_analise["Loja"].unique()** revela as 8 cidades da base.

Esse é o padrão geral do Pandas para operações numa coluna específica: nome da coluna, ponto, nome da operação. **.sum()** soma, **.count()** conta valores, **.value_counts()** conta quantas vezes cada valor aparece.

Módulo 3 – Encadeamento, np.where() e .map() (4 / 4)

```python
dic_regioes = {
    "Sao Paulo": "Sudeste", "Belo Horizonte": "Sudeste",
    "Rio De Janeiro": "Sudeste", "Online": "Online",
    "Salvador": "Nordeste", "Recife": "Nordeste",
    "Curitiba": "Sul", "Porto Alegre": "Sul",
}

df_analise["Regiao"] = df_analise["Loja"].map(dic_regioes)
```

Mapeamento com Dicionário e .map()

Com as 8 cidades em mãos, a terceira forma de criar colunas é o **mapeamento**: para cada valor de entrada, existe um valor de saída correspondente — exatamente o que um **dicionário** faz.

O método **.map()** aplica esse de-para direto na coluna: **df_analise["Loja"].map(dic_regioes)** — para cada loja, ele busca a chave correspondente no dicionário e preenche o valor.

O **.map()** também aceita uma **função** no lugar do dicionário, para mapeamentos mais complexos (com lógica condicional dentro). Para um de-para simples como este, passar o dicionário direto é mais rápido de escrever.

Módulo 3 – isna(), .loc e .iloc (1 / 3)

```
display(df_analise.isna().sum())
```

```
ID_Pedido          0
Data               0
Loja               0
Produto            0
Preco_Unitario     0
Qtd                0
Cliente            0
Faturamento        0
Forma_de_Venda     0
Região             0
dtype: int64
```

isna().sum(): Conferindo se o Mapeamento Funcionou

Com as três colunas novas criadas (**Faturamento**, **Forma_de_Venda**, **Região**), vale conferir se o **.map()** realmente funcionou para todas as lojas — se alguma não bater com o dicionário, a região daquela linha fica vazia.

Dá para rodar o **info()** de novo, mas para ver só a quantidade de vazios existe algo mais direto: **isna()** (é vazio?) aplicado na base inteira, seguido de **sum()** para somar os vazios de cada coluna.

Ao lado, o resultado: zero em todas as colunas, inclusive **Região** — confirmando que o mapeamento cobriu 100% das lojas. Esse é o mesmo padrão de aplicar uma função na base inteira (por coluna) em vez de numa coluna só.

Módulo 3 – isna(), .loc e .iloc (2 / 3)

.loc: Localizando por Nome da Coluna e da Linha

Com o tratamento e a engenharia de colunas prontos, começa a etapa de análise — e a primeira habilidade é **filtrar**: pegar só o pedaço da base que interessa.

Para localizar uma informação específica (uma linha, uma coluna), existe o **.loc**: localiza pelo **nome do índice da linha** e pelo **nome da coluna** — nesse caso, o índice é o mesmo número que aparece do lado esquerdo da tabela.

Ao lado: para descobrir a loja, o produto e o cliente do pedido de índice 3, basta **df_analise.loc[3, "Loja"]** (e o mesmo para as outras colunas) — o resultado confirma: Rio de Janeiro, Mouse Gamer, Cliente_17343.

```
# analisar -> filtrar

#.loc: id do pedido
id_pedido = 4

loja = df_analise.loc[3, "Loja"]
produto = df_analise.loc[3, "Produto"]
cliente = df_analise.loc[3, "Cliente"]
print(loja, produto, cliente)

#.iloc

# condicional

# exportar pedaços da base
# df_vendas_sp =
```

✓ 0.0s

```
Rio De Janeiro Mouse Gamer Cliente_17343
```

Módulo 3 – isna(), .loc e .iloc (3 / 3)

```python
# analisar -> filtrar

#.loc: id do pedido -> por nome da coluna e por nome da linha
id_pedido = 4

loja = df_analise.loc[3, "Loja"]
produto = df_analise.loc[3, "Produto"]
cliente = df_analise.loc[3, "Cliente"]
print(loja, produto, cliente)

#.iloc -> por posicao
id_pedido = 4
loja = df_analise.iloc[0, 1]

print(loja)
```

.loc vs .iloc: Nome do Índice ou Posição?

O **.iloc** também localiza uma linha e uma coluna, mas de um jeito diferente: em vez do nome do índice e da coluna, ele usa sempre a **posição** — a primeira linha é sempre 0, a segunda é 1, independente do que estiver escrito no índice.

Por enquanto, com a base na ordem original, índice e posição coincidem. Ao lado, **iloc[0, 1]** busca a linha na posição 0 e a coluna na posição 1 — nesse caso, a coluna **Data** — e dá o mesmo resultado que **loc[0, "Data"]** daria. Mas assim que a base for ordenada (próxima aula), os índices ficam embaralhados e só o **.iloc** continua contando pela posição.

Na prática, o **.loc** é disparado o mais usado — o **.iloc** ajuda em casos pontuais, como pegar sempre a última linha da base (**índice -1**) sem precisar saber o nome dela.

Módulo 3 – sort_values(), Filtro Condicional e to_csv() (1 / 4)

```
# analisar -> filtrar

df_analise = df_analise.sort_values(by="Data")
df_analise = df_analise.reset_index(drop=True)
```

sort_values() e reset_index(): Ordenando a Base

Para ordenar a base de acordo com uma coluna, existe o **sort_values()**, passando o parâmetro **by** com o nome da coluna de referência — aqui, **df_analise.sort_values(by="Data")** ordena tudo por data.

O efeito colateral: os índices ficam fora de ordem, porque cada linha carrega o índice que tinha antes de ordenar. Para recomeçar a contagem do zero, usa-se **reset_index()**.

Passando **drop=True**, o índice antigo é descartado — sem isso, ele viraria uma coluna nova na base, o que raramente é o que você quer.

Módulo 3 – sort_values(), Filtro Condicional e to_csv() (2 / 4)

Ordenando por Várias Colunas e o Parâmetro ascending

O by também aceita uma lista de colunas: by=["Data", "Faturamento"] ordena primeiro por data e, dentro de cada data, por faturamento — do menor para o maior.

Por padrão o sort_values() já ordena em ordem crescente — passar ascending=True explicitamente (como ao lado) não muda o resultado, só deixa claro pra quem lê o código. Para inverter — data mais recente e maior faturamento primeiro — basta trocar para ascending=False.

Vale lembrar: depois de ordenar, o .loc continua funcionando normal (busca pelo nome do índice), mas o .iloc passa a dar respostas diferentes, porque a posição das linhas mudou.

```python
# analisar -> filtrar

df_analise = df_analise.sort_values(by=["Data", "Faturamento"])
df_analise = df_analise.reset_index(drop=True)

# analisar -> filtrar

df_analise = df_analise.sort_values(by=["Data", "Faturamento"], ascending=True)
df_analise = df_analise.reset_index(drop=True)
```

Módulo 3 – sort_values(), Filtro Condicional e to_csv() (3 / 4)

```
# condicional
df_id_pedido_4 = df_analise[df_analise["ID_Pedido"]==4]

# condicional
# df_id_pedido_4 = df_analise[df_analise["ID_Pedido"]==4]

# exportar pedaços da base
df_vendas_sp = df_analise[df_analise["Loja"]=="São Paulo"]

display(df_vendas_sp)
```

✓ 0.0s

```
Rio De Janeiro
Mouse Gamer
Cliente_17343
Recife Cabo HDMI Cliente_11523
```

	ID_Pedido	Data	Loja	Produto	Preco_Unitario	Qtd	Client
19	39483	2023-01-01	São Paulo	Mouse Gamer	120.0	1	Cliente_127
21	48375	2023-01-01	São Paulo	Mouse Gamer	120.0	1	Cliente_154
23	90270	2023-01-01	São Paulo	Mouse Gamer	120.0	1	Cliente_641
		2023-	São	Teclado			

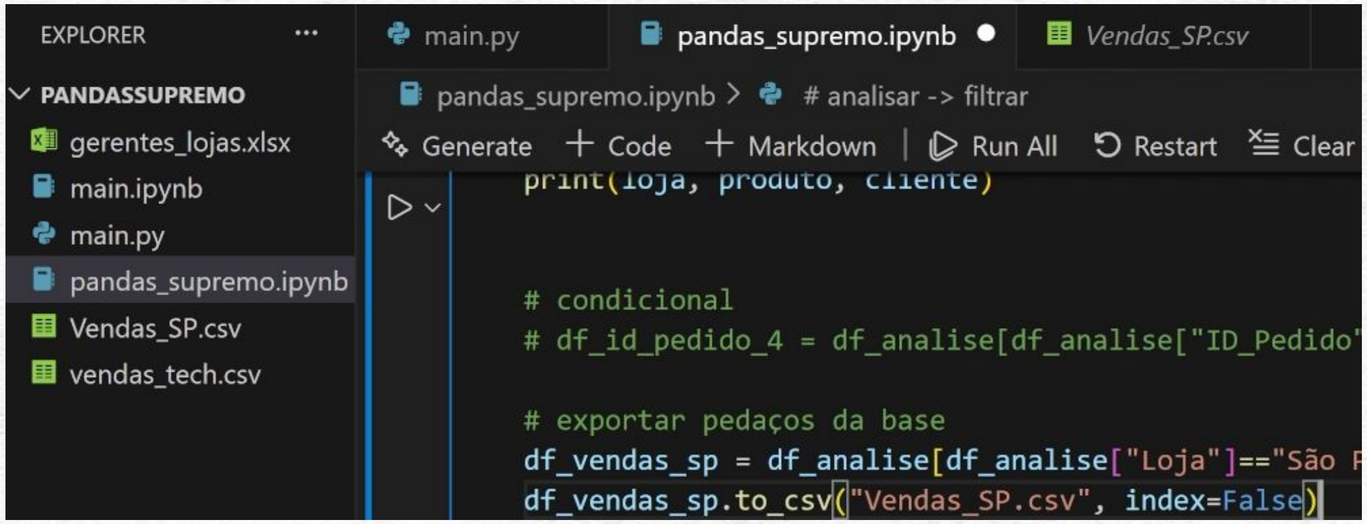
Filtro Condicional: a Forma Mais Usada de Filtrar

A quarta e mais usada forma de filtrar é a **condicional**: em vez de um índice fixo (**.loc**/**.iloc**), você passa uma condição que vale para qualquer linha — igual a, maior que, dentro de uma lista, e por aí vai.

A sintaxe: **df_analise[df_analise["ID_Pedido"] == 4]** — o Pandas aplica a condição em toda a coluna e devolve uma **tabela nova** só com as linhas onde ela é verdadeira.

Ao lado, o mesmo padrão para criar uma sub-base só com as vendas de **São Paulo**: **df_vendas_sp = df_analise[df_analise["Loja"] == "São Paulo"]**. Repare que os índices do resultado (19, 21, 23, 37...) continuam sendo os originais — o filtro não reordena nada.

Módulo 3 – sort_values(), Filtro Condicional e to_csv() (4 / 4)



Exportando com to_csv()

Depois de filtrar uma sub-base (como a de São Paulo), pode ser que você precise mandar ela para alguém ou usar fora do notebook. Para isso existe o **.to_csv()**.

df_vendas_sp.to_csv("Vendas_SP.csv", index=False) gera o arquivo — repare o parâmetro **index=False**: sem ele, a coluna de índice (19, 21, 23...) viraria uma coluna a mais no CSV, o que quase nunca é o que você quer.

Ao lado, o arquivo **Vendas_SP.csv** já aparece na barra lateral, criado automaticamente. Se exportar de novo com o mesmo nome, ele simplesmente substitui o anterior. O mesmo método funciona para exportar em Excel, só trocando para **.to_excel()**.

Módulo 3 – Filtro por Data, Condições Múltiplas e groupby() (1 / 3)

```
# exportar as vendas de 2024
df_vendas_2024 = df_analise[df_analise["Data"].dt.year==2024]

display(df_vendas_2024)
```

✓ 0.2s

```
Rio De Janeiro
Mouse Gamer
Cliente_17343
Porto Alegre Cabo HDMI Cliente_22886
```

	ID_Pedido	Data	Loja	Produto	Preco_Unitario	Qtd	Cliente
50423	22735	2024-01-01	São Paulo	Cabo HDMI	40.0	1	Cliente_14772

Filtrando por Data: .dt.year

E se você precisar de uma base só com as vendas de um determinado ano? A coluna **Data** já é do tipo data, mas não dá pra comparar direto com "2024" — é preciso extrair só o ano dela.

Assim como **.str** libera fórmulas de texto, **.dt** libera fórmulas de data: **df_analise["Data"].dt.year** devolve o ano de cada linha, pronto para comparar.

Ao lado: **df_analise[df_analise["Data"].dt.year == 2024]**. Outra forma, sem precisar de **.dt**, é comparar a data inteira como texto no formato ano-mês-dia: **df_analise["Data"] >= "2024-01-01"** — ambas chegam ao mesmo resultado.

Módulo 3 – Filtro por Data, Condições Múltiplas e groupby() (2 / 3)

Combinando Condições: & e |

Para filtrar por mais de uma condição ao mesmo tempo — por exemplo, só vendas de **Cabo HDMI** na região **Sul** — combine duas condições com **&** (e comercial, "as duas") ou **|** (barra vertical, "uma ou outra, ou as duas").

Ao lado: duas condições guardadas em variáveis separadas (**condicao**, **condicao2**) e combinadas com **&** dentro do filtro — mais fácil de ler do que escrever tudo numa linha só.

Se escrever as condições direto dentro dos colchetes (sem variável), **sempre coloque cada uma entre parênteses**. Sem isso, o Pandas pode não entender onde uma condição termina e a outra começa, e o erro que aparece costuma ser bem confuso de rastrear.

```
# duplas condições
condicao = df_analise["Produto"]=="Cabo HDMI"
condicao2 = df_analise["Região"]=="Sul"
df_vendas_HDMI_SUL = df_analise[condicao & condicao2]

display(df_vendas_HDMI_SUL)
```

✓ 0.2s

```
Rio De Janeiro
Mouse Gamer
Cliente_17343
Porto Alegre Cabo HDMI Cliente_22886
```

	ID_Pedido	Data	Loja	Produto	Preco_Unitario	Qtd	Clien
0	1700	2023-01-01	Porto Alegre	Cabo HDMI	40.0	1	Cliente_144
2	35599	2023-01-01	Porto Alegre	Cabo HDMI	40.0	1	Cliente_186
3	37130	2023-01-	Porto	Cabo	40.0	1	Cliente_228

Módulo 3 – Filtro por Data, Condições Múltiplas e groupby() (3 / 3)

```
analise_lojas = df_analise[["Loja", "Faturamento"]].groupby("Loja").sum()
```

Loja	Faturamento
Belo Horizonte	41714890.0
Curitiba	41121720.0
Online	6080850.0
Porto Alegre	41678460.0
Recife	42190390.0
Rio De Janeiro	42294720.0
Salvador	42300610.0
São Paulo	42090690.0

groupby(): Sua Primeira Análise por Agrupamento

Chegou a etapa que mais lembra "análise de dados" de verdade: agrupamentos. A primeira pergunta: **qual loja vendeu mais?**

O **groupby()** agrupa uma coluna — cada valor único aparece uma única vez no resultado. Mas ele só funciona se você disser o que fazer com as outras colunas: aplicar o **groupby()** na base inteira sem selecionar colunas dá erro, porque ele tenta somar até colunas de texto.

A solução: selecionar só as colunas que interessam antes de agrupar. Ao lado, **df_analise[["Loja", "Faturamento"]].groupby("Loja").sum()** agrupa por loja e soma o faturamento de cada uma — a coluna agrupada (**Loja**) vira o novo índice do resultado.

Módulo 3 – groupby() Avançado (1 / 4)

Ordenando e Formatando o Resultado do groupby()

O resultado de um **groupby()** é uma tabela normal — dá para ordenar com **sort_values()** igual antes, usando **ascending=False** para o maior faturamento aparecer primeiro.

Por padrão, o **groupby()** transforma a coluna agrupada (**Loja**) em índice. Boa prática: manter textos como colunas, não como índice — por isso, **reset_index()** (sem **drop=True**, porque dessa vez o índice antigo é útil).

Por fim, formatação visual com **.map()**: **"R${:,.2f}".format** aplica o código de formatação (separador de milhar + 2 casas decimais) em cada valor. Cuidado: depois disso a coluna vira texto — só faça isso quando a análise já estiver pronta.

```python
# ranking de faturamento por loja
analise_lojas = df_analise[["Loja", "Faturamento"]].groupby("Loja").sum()
analise_lojas = analise_lojas.sort_values(by="Faturamento", ascending=False)
analise_lojas = analise_lojas.reset_index()
analise_lojas["Faturamento"] = analise_lojas["Faturamento"].map("R${:,.2f}".format)
```

	Loja	Faturamento
0	Salvador	R$42,300,610.00
1	Rio De Janeiro	R$42,294,720.00
2	Recife	R$42,190,390.00
3	São Paulo	R$42,090,690.00

```
analise_produtos_online = df_vendas_online[["Produto", "Faturamento"]].groupby("Produto").sum()
analise_produtos_online = analise_produtos_online.sort_values(by="Faturamento", ascending=False,
display(analise_produtos_online)

display(analise_lojas)
```

✓ 0.0s

Produto	Faturamento
iPhone 14	2145000.0
Notebook HP	1414400.0
Notebook Dell	1291500.0
Smartphone Samsung	684200.0
Monitor 27"	398400.0
Teclado Mecânico	85750.0
Mouse Gamer	45480.0
Cabo HDMI	16120.0

groupby() Depois de Filtrar: Produtos Mais Vendidos Online

Análise mais elaborada: quais produtos mais venderam **nas vendas online**? O primeiro passo é filtrar — igual você já aprendeu — e só depois agrupar o resultado filtrado.

Primeiro filtra: **df_vendas_online = df_analise[df_analise["Loja"] == "Online"]**. Depois agrupa esse resultado por produto, soma o faturamento e ordena decrescente — o mesmo padrão de sempre, só que aplicado em cima de uma base já filtrada.

Ao lado, o ranking: **iPhone 14** lidera o faturamento online, seguido de Notebook HP e Notebook Dell. Trocar **Faturamento** por **Qtd** no agrupamento dá o ranking por quantidade em vez de valor — cada pergunta pede uma métrica diferente.

Módulo 3 – groupby() Avançado (3 / 4)

rename(): Renomeando Colunas para Apresentação

Nomes técnicos como **Qtd** funcionam bem no código, mas não numa apresentação para o chefe. Para trocar o nome de uma coluna, existe o **rename()**.

A sintaxe usa um dicionário: **.rename(columns={"Qtd": "Vendas Totais"})** — a chave é o nome atual, o valor é o nome novo. Dá para renomear quantas colunas quiser de uma vez, todas no mesmo dicionário.

Se você errar o nome da coluna atual (uma que não existe na base), o Pandas simplesmente ignora — não dá erro, só não muda nada. Esse mesmo recurso também é muito usado logo na importação, para já padronizar nomes de colunas bagunçados vindos de uma planilha ou API.

```
df_vendas_online = df_analise[df_analise["Loja"]=="Online"]
analise_produtos_online = df_vendas_online[["Produto", "Qtd"]].groupby("Produto").sum(
analise_produtos_online = analise_produtos_online.sort_values(by="Qtd", ascending=Fals
analise_produtos_online = analise_produtos_online.rename(columns={"Qtd": "Vendas Totai
display(analise_produtos_online)
```

✓ 0.0s

	Loja	Faturamento
0	Salvador	R$42,300,610.00
1	Rio De Janeiro	R$42,294,720.00
2	Recife	R$42,190,390.00
3	São Paulo	R$42,090,690.00
4	Belo Horizonte	R$41,714,890.00
5	Porto Alegre	R$41,678,460.00
6	Curitiba	R$41,121,720.00
7	Online	R$6,080,850.00

Produto	Vendas Totais
Notebook HP	442
Cabo HDMI	403

Módulo 3 – groupby() Avançado (4 / 4)

```python
analise_produtos_em_loja = df_analise[["Loja", "Produto", "Qtd"]]
analise_produtos_em_loja = analise_produtos_em_loja.groupby(
    ["Loja", "Produto"]
).sum()
```

		Qtd
Loja	**Produto**	
Belo Horizonte	Cabo HDMI	2636
	Monitor 27"	2625
	Mouse Gamer	2465
	Notebook Dell	2654
	Notebook HP	2775
...	...	...
São Paulo	Notebook Dell	2535

Agrupando por Duas Colunas: Loja e Produto

E se a pergunta for mais específica — quais produtos mais venderam **em cada loja**? Isso exige agrupar por duas regras ao mesmo tempo, e a **ordem** delas importa.

Em vez de passar um nome só para o **groupby()**, passe uma **lista**: **groupby(["Loja", "Produto"])**. Ele agrupa primeiro por loja e, dentro de cada loja, quebra por produto — o resultado vira um índice de dois níveis.

Ao lado, o resultado: Belo Horizonte aparece com todos os seus produtos, depois a próxima loja, e assim por diante. Repare o **"..."** no meio — o display do Pandas oculta linhas do meio quando a tabela é grande. Resolver isso é o próximo passo.

Módulo 3 – Exibir Tabela Completa e merge() (1 / 3)

```
# analise de rankind por loja e por produto
# quais produtos venderam mais em cada uma das lojas
analise_produtos_em_lojas = df_analise[["Loja", "Produto", "Qtd"]].groupby(["Loja", "Produto"]).s

with pd.option_context("display.max_rows", None):
    display(analise_produtos_em_lojas)
```

Mostrando a Tabela Inteira: option_context

Quando você agrupa por duas colunas (como loja e produto), o resultado pode ter muitas linhas — e o Pandas, por padrão, oculta o meio da tabela com um "..." para não poluir a tela.

Para ver a tabela inteira só naquela linha específica (sem mudar o comportamento do resto do notebook), use **with pd.option_context("display.max_rows", None):** — tudo que estiver identado dentro desse bloco mostra sem limite de linhas; fora dele, volta ao padrão.

Use com moderação: em tabelas muito grandes (muitas linhas ou colunas), isso pode travar o editor. Vale só para conferências pontuais.

Módulo 3 – Exibir Tabela Completa e merge() (2 / 3)

Preparando para o Merge: Filtrar, Agrupar, as_index=False

Próxima análise: quais gerentes bateram a meta em janeiro de 2023? Essa informação está espalhada em **duas bases diferentes** — as vendas (**df_analise**) e os gerentes de cada loja (**df_gerentes**, carregada lá no início).

Primeiro, filtra só janeiro de 2023 combinando duas condições com **&**: **(dt.year==2023) & (dt.month==1)**. Depois, agrupa por loja e soma o faturamento — mas dessa vez com **as_index=False**, porque a coluna Loja vai ser usada daqui a pouco para juntar com a outra base, e não pode virar índice.

Regra prática: quando a informação de fora só serve para uma correspondência final (não para o agrupamento em si), traga-a **depois** de agrupar — dá menos trabalho do que trazer tudo antes.

```
display(df_analise.head())

# quais gerentes bateram a meta em janeiro de 2023
df_meta = df_analise[(df_analise["Data"].dt.year==2023) & (df_analise["Data"].dt.month==1)]

df_meta = df_meta[["Loja", "Faturamento"]].groupby("Loja", as_index=False).sum()

display(df_meta)
```

Módulo 3 – Exibir Tabela Completa e merge() (3 / 3)


```python
# quais gerentes bateram a meta em janeiro de 2023
df_meta = df_analise[(df_analise["Data"].dt.year==2023) & (df_analise["Data"].dt.month==1)]

df_meta = df_meta[["Loja", "Faturamento"]].groupby("Loja", as_index=False).sum()

df_meta = df_meta.merge(df_gerentes, on="Loja")

display(df_meta)
```

merge(): Juntando Duas Bases com on

Para juntar duas bases de dados no Pandas, existem dois métodos bem diferentes: **merge()** (juntar colunas — como uma chave estrangeira de banco de dados) e **concat()** (empilhar linhas — bases com as mesmas colunas, uma embaixo da outra). Aqui, o caso é merge.

A sintaxe: **df_meta.merge(df_gerentes, on="Loja")**. O parâmetro **on** diz qual coluna usar para fazer a correspondência entre as duas tabelas — funciona quando o nome da coluna é **igual** nas duas.

Se os nomes forem diferentes (ex.: "Loja" numa base, "Cidade" na outra), use **left_on** e **right_on** — "esquerda" é a tabela antes do **.merge(**, "direita" é a que está dentro dos parênteses.

Módulo 3 – Tipos de Join, Bateu Meta e Série Temporal (1 / 3)

Tipos de Join: left vs inner

Por padrão, **merge()** faz um **inner join**: só inclui no resultado as lojas que existem **nas duas** tabelas. Se uma loja não tiver gerente cadastrado, ela simplesmente **desaparece** da tabela final — sem aviso nenhum.

Passando **how="left"**, o resultado garante **todas** as linhas da tabela da esquerda (**df_meta**), preenchendo com vazio (**NaN**) onde não achar correspondência — bem mais seguro para conferir o que não bateu.

Ao lado: com **left join**, **Online** (sem gerente mesmo) e **Rio De Janeiro** (só um problema de nome despadronizado entre as duas bases) aparecem com **NaN** — o que denuncia exatamente onde investigar.

```
df_meta = df_meta.merge(
    df_gerentes, on="Loja", how="left"
)
```

	Loja	Faturamento	Gerente	Meta_Mensal
0	Belo Horizonte	1779100.0	Juliana	55000.0
1	Curitiba	1986920.0	Roberto	45000.0
2	Online	404570.0	NaN	NaN
3	Porto Alegre	1726640.0	Pedro	42000.0
4	Recife	1779020.0	Marcos	48000.0
5	Rio De Janeiro	1736830.0	NaN	NaN
6	Salvador	1686070.0	Ana	52000.0
7	São Paulo	1831140.0	Carlos	50000.0

Módulo 3 – Tipos de Join, Bateu Meta e Série Temporal (2 / 3)

```
df_meta["Bateu Meta"] = np.where(
    df_meta["Faturamento"] >= df_meta["Meta_Mensal"],
    "Sim", "Nao"
)
```

	Loja	Faturamento	Gerente	Meta_Mensal	Bateu Meta
0	Belo Horizonte	1779100.0	Juliana	55000.0	Sim
1	Curitiba	1986920.0	Roberto	45000.0	Sim
2	Online	404570.0	NaN	NaN	Não
3	Porto Alegre	1726640.0	Pedro	42000.0	Sim
4	Recife	1779020.0	Marcos	48000.0	Sim
5	Rio De Janeiro	1736830.0	Fernanda	60000.0	Sim
6	Salvador	1686070.0	Ana	52000.0	Sim
7	São Paulo	1831140.0	Carlos	50000.0	Sim

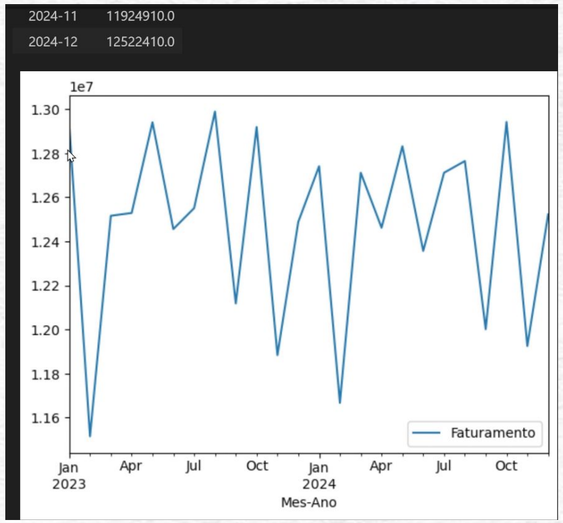
Corrigindo a Correspondência e a Coluna Bateu Meta

O Rio De Janeiro sem gerente era um bug de padronização: a base de gerentes tinha o nome da loja escrito diferente da base de vendas. A correção: aplicar o mesmo .str.strip().str.title() que já era usado em df_analise também na base df_gerentes — e rodar tudo de novo (Run All).

Esse é o padrão de qualquer análise: você antecipa o máximo de tratamento possível, mas alguns problemas só aparecem quando o cruzamento de bases já está pronto.

Com as bases corrigidas, dá para responder a pergunta original: quais gerentes bateram a meta? Basta um np.where() comparando Faturamento com Meta_Mensal — ao lado, o resultado final: todo mundo bateu, menos o Online (que nem tem meta definida).

Módulo 3 – Tipos de Join, Bateu Meta e Série Temporal (3 / 3)



Série Temporal: to_period() e .plot()

Última análise: como o faturamento evolui **mês a mês**, ao longo do tempo? Diferente das anteriores, essa é sensível à data — não basta agrupar uma característica, é preciso pensar em tendência.

Como a coluna **Data** já é do tipo data, dá para criar uma coluna de período com **df_analise["Data"].dt.to_period("M")** — agrupa automaticamente por mês e ano (**"2023-01"**, **"2023-02"**...). Depois, é só agrupar por essa coluna e somar o faturamento, igual sempre.

O Pandas já vem com gráficos do Matplotlib embutidos: **.plot()** numa tabela com índice de data gera o gráfico de linha automaticamente, já ordenado no tempo — como ao lado. Para gráficos mais bonitos e interativos, vale migrar para o Plotly depois de dominar o básico com Pandas.

```
dados = {
    "Nome": ["Ana", "Bruno", "Carlos", "Daniela", "Eduardo"],
    "Idade": [25, 30, 35, 40, 45],
    "Salario": [3000, None, 7000, None, 11000]
}
tabela = pd.DataFrame(dados)
```

5 Funções Essenciais do Pandas: uma Revisão Rápida

Essa aula é uma revisão direta ao ponto: em vez da base de vendas que a gente usou no módulo inteiro, o exemplo aqui é bem menor — uma tabelinha de funcionários — só para você rever, de forma isolada, 5 funções que aparecem o tempo todo em qualquer projeto de dados.

Ao lado, a base de partida: um dicionário Python com **Nome**, **Idade** e **Salario** de 5 funcionários, virando um DataFrame com **pd.DataFrame(dados)**. Repare que dois salários estão como **None** — de propósito, para você já ter valores vazios para tratar mais adiante.

Detalhe prático: o nome da coluna aqui é **Salario** sem acento. Não é erro — é recomendação: evite acento e espaço em nome de coluna, porque facilita muito na hora de escrever o código depois (menos chance de digitar errado, menos problema de codificação).

Módulo 3 – Revisão das Funções do Pandas (2 / 6)

1. describe(): Resumo Estatístico Rápido

A primeira função é a que você já viu no módulo inteiro: **describe()**. Aplicado direto na tabela, ele calcula automaticamente um resumo estatístico de toda coluna numérica.

Ao lado: **tabela.describe()** traz **count** (quantos valores preenchidos — repare que **Salario** mostra só 3, porque os dois vazios não entram na conta), **mean** (média), **std** (desvio padrão), **min**, **max** e os quartis.

É sempre o primeiro comando para você ter uma ideia rápida da faixa de valores de uma base nova — antes de decidir qualquer tratamento.

```
print(tabela.describe())
```

	Idade	Salario
count	5.000000	3.0
mean	35.000000	7000.0
std	7.905694	4000.0
min	25.000000	3000.0
25%	30.000000	5000.0
50%	35.000000	7000.0
75%	40.000000	9000.0
max	45.000000	11000.0

```
tabela["Salario Ajustado"] = tabela["Salario"].apply(
    lambda x: x * 1.1
)
```

	Nome	Idade	Salario	Salario Ajustado
0	Ana	25	3000.0	3300.0
1	Bruno	30	NaN	NaN
2	Carlos	35	7000.0	7700.0
3	Daniela	40	NaN	NaN
4	Eduardo	45	11000.0	12100.0

2. apply(): Aplicando uma Função em Cada Valor

A segunda função é o **.apply()**: ele pega uma coluna inteira e aplica uma função em **cada valor** dela, um por um, devolvendo uma coluna nova com os resultados.

Ao lado: **tabela["Salario"].apply(lambda x: x * 1.1)** cria a coluna **Salario Ajustado**, multiplicando cada salário por 1,1 (um reajuste de 10%). A **lambda** é só uma função sem nome, direto na linha — o **x** representa o valor de cada célula da coluna.

Repare: como Bruno e Daniela tinham **Salario** vazio, o resultado do **apply()** para eles também sai vazio (**NaN**) — não dá para multiplicar um valor que não existe. Isso é exatamente o motivo de tratar vazios antes de seguir com a análise.

Módulo 3 – Revisão das Funções do Pandas (4 / 6)

3. fillna(): Preenchendo os Valores Vazios

A terceira função resolve o problema que acabou de aparecer: **.fillna()** preenche valores vazios com o que você passar para ele.

Ao lado: primeiro calcula a média dos salários preenchidos (**tabela["Salario"].mean()**, que dá 7000 — a média de 3000, 7000 e 11000), e usa esse valor para preencher os vazios com **fillna(media_salarial)**. Depois, repete o mesmo padrão para **Salario Ajustado**, preenchendo com a própria média da coluna.

Resultado: Bruno e Daniela agora aparecem com 7000 e 7700 — não é o salário real deles, é uma estimativa razoável para a análise não quebrar. Preencher com a média é só uma entre várias estratégias possíveis; a escolha certa depende do contexto de cada análise.

```
media_salarial = tabela["Salario"].mean()
tabela["Salario"] = tabela["Salario"].fillna(media_salarial)
tabela["Salario Ajustado"] = tabela["Salario Ajustado"].fillna(
    tabela["Salario Ajustado"].mean()
)
```

	Nome	Idade	Salario	Salario Ajustado
0	Ana	25	3000.0	3300.0
1	Bruno	30	7000.0	7700.0
2	Carlos	35	7000.0	7700.0
3	Daniela	40	7000.0	7700.0
4	Eduardo	45	11000.0	12100.0

```python
tabela_areas = pd.DataFrame({
    "Nome Funcionario": ["Ana", "Bruno", "Carlos", "Eduardo"],
    "Area": ["Administrativo", "Financeiro", "Vendas", "Financeiro"]
})

tabela = tabela.merge(
    tabela_areas, how="left",
    left_on="Nome", right_on="Nome Funcionario"
)
tabela = tabela.drop(columns="Nome Funcionario")
```

	Nome	Idade	Salario	Salario Ajustado	Area
0	Ana	25	3000.0	3300.0	Administrativo
1	Bruno	30	7000.0	7700.0	Financeiro
2	Carlos	35	7000.0	7700.0	Vendas
3	Daniela	40	7000.0	7700.0	NaN
4	Eduardo	45	11000.0	12100.0	Financeiro

4. merge(): Juntando com uma Segunda Tabela

A quarta função é o **merge()**, que você já viu bastante no módulo: juntar duas tabelas usando uma coluna em comum como referência.

Ao lado, uma segunda tabela — **tabela_areas** — traz a área de cada funcionário, mas com um detalhe: a coluna de nome se chama **Nome Funcionario** ali, não **Nome** como na tabela original. Por isso o merge usa **left_on="Nome"** e **right_on="Nome Funcionario"** em vez de um simples **on**.

Com **how="left"**, todo mundo da tabela original é mantido — e **Daniela**, que não está em **tabela_areas**, fica com **Área** vazia (**NaN**), exatamente o comportamento do left join que você já conhece. No final, **drop(columns="Nome Funcionario")** tira a coluna duplicada de nome.

Módulo 3 – Revisão das Funções do Pandas (6 / 6)

5. groupby(): Agrupando por Área — Só Depois do Merge

A quinta e última função é o **groupby()**, para agrupar os dados por uma característica em comum — nesse caso, a **Área** de cada funcionário.

Repare na ordem: essa é a última função da revisão de propósito, porque a coluna **Área** só existe depois do **merge()** do slide anterior. Tentar agrupar por uma coluna antes dela existir daria erro — sequência importa em Pandas.

Ao lado: **tabela.groupby("Area").mean(numeric_only=True)** calcula a média de idade e salário por área. O parâmetro **numeric_only=True** evita erro tentando calcular média de colunas de texto. Financeiro aparece com a maior média salarial (9000), porque reúne Bruno e Eduardo.

```
medias_area = tabela.groupby("Area").mean(
    numeric_only=True
)
```

Area	Idade	Salario	Salario Ajustado
Administrativo	25.0	3000.0	3300.0
Financeiro	37.5	9000.0	9900.0
Vendas	35.0	7000.0	7700.0

Módulo 3 – [Opcional] Aula de Orientação sobre o Projeto Exercício de Revisão (1 / 1)

Como Funciona o Projeto Exercício de Revisão

Esta aula é **opcional**. Ela serve só para explicar o que vem a seguir na **Aula 16** — o Projeto Exercício de Revisão — antes de você decidir se quer fazê-lo.

O que é o exercício? Nas duas partes da Aula 16, você vai acompanhar o professor reconstruindo, **do zero**, exatamente o mesmo projeto de análise de dados que você já estudou ao longo deste módulo — a mesma base de vendas, o mesmo desafio de negócio proposto pelo CEO, os mesmos passos de tratamento e análise.

Não é conteúdo novo. Todas as funções, parâmetros e técnicas que aparecem já foram ensinadas nas aulas anteriores. O objetivo aqui é **repetição espaçada**: ver o processo inteiro rodar de novo, do início ao fim, ajuda a fixar de um jeito diferente de simplesmente reler os slides.

A estrutura tem duas partes. A **Parte 1** cobre ETL, carregamento das bases, inspeção e todo o tratamento de dados — colunas, valores vazios, tipos, padronização de texto e duplicatas. A **Parte 2** cobre a engenharia de colunas (faturamento, forma de venda, região), filtros, agrupamentos, junção com a base de gerentes e a análise de série temporal.

Como aproveitar melhor: tente acompanhar digitando o código você mesmo, em vez de só assistir. Se em algum ponto você sentir firmeza total no que está sendo feito, sinta-se à vontade para pular adiante — o exercício existe para reforçar quem quer mais prática, não é pré-requisito para o restante do curso.

```
Requirement already satisfied: pandas in c:\users\joaol\appdata\local\programs\python\python311\lib\site-packages (2
.2.0)
Requirement already satisfied: openpyxl in c:\users\joaol\appdata\local\programs\python\python311\lib\site-packages
(3.1.2)
Requirement already satisfied: numpy<2,>=1.23.2 in c:\users\joaol\appdata\local\programs\python\python311\lib\site-p
ackages (from pandas) (1.26.3)
Requirement already satisfied: python-dateutil>=2.8.2 in c:\users\joaol\appdata\local\programs\python\python311\lib\
site-packages (from pandas) (2.8.2)
Requirement already satisfied: pytz>=2020.1 in c:\users\joaol\appdata\local\programs\python\python311\lib\site-packa
ges (from pandas) (2023.3.post1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\joaol\appdata\local\programs\python\python311\lib\site-pac
kages (from pandas) (2023.4)
Requirement already satisfied: et-xmlfile in c:\users\joaol\appdata\local\programs\python\python311\lib\site-package
s (from openpyxl) (1.1.0)
Requirement already satisfied: six>=1.5 in c:\users\joaol\appdata\local\programs\python\python311\lib\site-packages
(from python-dateutil>=2.8.2->pandas) (1.16.0)
```

Pandas: a Ferramenta de ETL do Python

Esse projeto é um exercício de revisão: a proposta é refazer, do zero, o mesmo tipo de análise que você já viu antes — só que agora acompanhando a construção completa, com todas as decisões explicadas de novo.

O **Pandas** é a ferramenta mais usada na área de dados em Python. Ele cobre o processo de **ETL** — Extract, Transform, Load: extrair os dados, transformá-los (corrigir problemas da base) e carregá-los para você fazer as análises direto ali dentro, sem precisar de outra ferramenta.

Ao lado: instalação com **pip install pandas openpyxl**. O Pandas não vem instalado por padrão — e o **openpyxl** é a engine que ele usa por trás dos panos para ler arquivos Excel. Cada formato de base pode exigir uma biblioteca própria: se faltar, o Pandas avisa na hora de carregar.

Por que .ipynb em vez de .py?

Em vez de um arquivo Python comum, o projeto usa um **notebook Jupyter** — extensão **.ipynb**. Analistas, cientistas e engenheiros de dados costumam trabalhar com esse formato, principalmente no início e durante o desenvolvimento de um projeto.

Ao lado: o nome do arquivo é **pandas_supremo.ipynb**, nunca **pandas.ipynb** — esse nome bateria com o da própria biblioteca e causaria conflito na hora de importar.

A diferença prática do notebook: você adiciona **células de código** (botão **+ Code**) e roda cada uma de forma independente — não precisa executar o script inteiro de novo a cada mudança, o que ajuda muito na hora de visualizar dados no meio do processo.

```
# arquivo do projeto
pandas_supremo.ipynb


# nao use: conflita com a lib
# pandas.ipynb
```

O Desafio do Projeto: Duas Bases de Dados

O desafio deste exercício: o CEO da empresa pediu um relatório de vendas por gerente — quanto cada gerente está vendendo, qual produto cada um mais vende — e uma leitura da tendência de faturamento mensal (subindo, caindo ou estável).

Para responder isso, o projeto usa **duas bases de dados**, cada uma num formato diferente — o mesmo cenário que você encontra no mercado de trabalho.

Ao lado, um pedaço real de **vendas_tech.csv** — a base de vendas em CSV. A segunda base é **gerentes_lojas.xlsx**, em Excel, com o gerente de cada loja. Ter as duas juntas força você a lidar com formatos diferentes, exatamente o que o Pandas resolve.

```
1   ID_Pedido,Data,Loja,Produto,Preco_Unitario,Qtd,Cliente,Data_Base
2   1,2023-06-08,São Paulo,Mouse Gamer,120.0,1,Cliente_4095,2025-01-01
3   2,2023-03-01,Belo Horizonte,iPhone 14,5500.0,1,Cliente_8750
4   3,2023-02-25,,"Monitor 27""",1200.0,1,Cliente_14859
5   4,2024-11-19,RIO DE JANEIRO,Mouse Gamer,120.0,2,Cliente_17343
6   5,2024-01-27,Rio de Janeiro,Smartphone Samsung,2200.0,1,Cliente_23377
7   6,2023-03-11,São Paulo,Cabo HDMI,40.0,1,Cliente_24809
8   7,2024-12-08,Salvador,"Monitor 27""",1200.0,1,Cliente_11076
9   8,2024-09-12,São Paulo,"Monitor 27""",1200.0,2,Cliente_14053
10  9,2023-05-20,Belo Horizonte,Mouse Gamer,120.0,1,Cliente_19175
11  10,2024-10-16,Recife,Notebook HP,3200.0,1,Cliente_13508
12  11,2023-06-28,RIO DE JANEIRO,Smartphone Samsung,2200.0,1,Cliente_3253
```

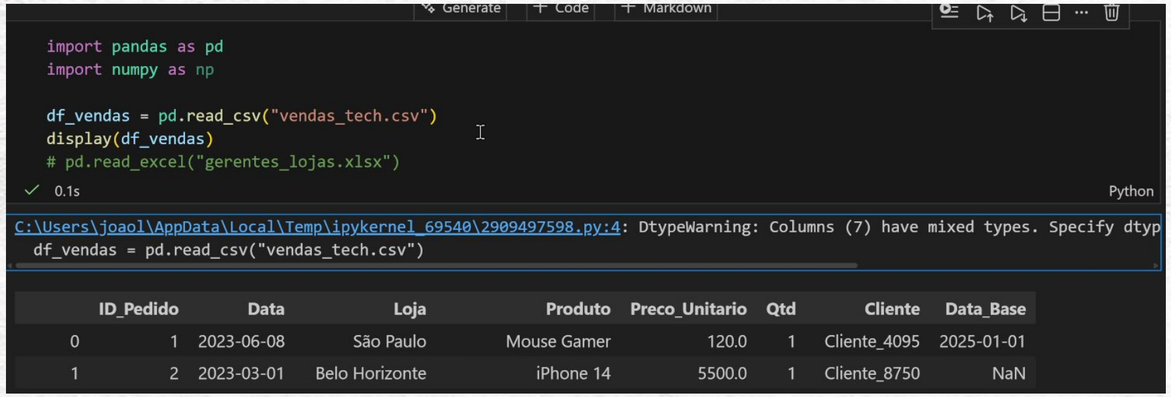


Importando as Bibliotecas e Carregando os Dados

Primeiro passo do código: importar as bibliotecas. O **Pandas** é sempre importado com o apelido **pd**, e junto dele quase sempre vem o **NumPy** como **np** — é ele quem faz, por trás do Pandas, as operações matriciais de forma muito eficiente.

Com as bibliotecas prontas, vem a primeira grande funcionalidade do Pandas: carregar as informações. O comando começa sempre com **pd.read_**, seguido do formato do arquivo — CSV, Excel, HTML, JSON, Parquet, SQL e por aí vai.

Ao lado: **pd.read_csv("vendas_tech.csv")** para a base de vendas e **pd.read_excel("gerentes_lojas.xlsx")** para a base de gerentes — cada formato usa o método correspondente, e por isso os dois já entram nomeados de forma intuitiva: **df_vendas** e **df_gerentes**.



	ID_Pedido	Data	Loja	Produto	Preco_Unitario	Qtd	Cliente	Data_Base
0	1	2023-06-08	São Paulo	Mouse Gamer	120.0	1	Cliente_4095	2025-01-01
1	2	2023-03-01	Belo Horizonte	iPhone 14	5500.0	1	Cliente_8750	NaN

DataFrame: a Tabela do Pandas

Quando o Pandas lê uma base de dados, ele guarda essa tabela dentro de uma estrutura que ele chama de **DataFrame** — a estrutura base do Pandas.

Boa prática: em vez de chamar a variável de **dataframe** por extenso, o mercado usa o prefixo **df_** seguido de um nome intuitivo — **df_vendas**, porque é a base de vendas. Só de ler o nome já dá para saber o que a variável guarda.

Ao lado: nos arquivos **.ipynb**, no lugar do **print()** existe o **display()** — mostra a mesma tabela de forma bem mais visual. O aviso **DtypeWarning** que aparece embaixo é só um alerta sobre colunas com tipos de dados misturados — a gente resolve isso já no próximo bloco.

encoding: Lidando com Caracteres Especiais

O pd.read_csv aceita vários parâmetros opcionais — dica: digite uma vírgula após o nome do arquivo dentro do read_csv(...) para o autocomplete listar todos eles.

O primeiro é o encoding: define como os caracteres especiais são interpretados na leitura. Bases com acento, cedilha ou til — muito comuns aqui no Brasil — às vezes dão erro de leitura por causa disso.

Por padrão o Pandas já usa encoding="utf-8". Quando dá erro, o mais comum é resolver com encoding="latin-1", como ao lado — o mais usado para bases com caracteres em português. Na dúvida, jogue a mensagem de erro numa IA.

```
df_vendas = pd.read_csv(
    "vendas_tech.csv",
    encoding="latin-1"
)
```

Generate + Code + Markdown

```
import pandas as pd
import numpy as np

df_vendas = pd.read_csv("vendas_tech.csv", sep=";")
display(df_vendas)
df_gerentes = pd.read_excel("gerentes_lojas.xlsx")
display(df_gerentes)
```

sep: o Separador de Colunas

O segundo parâmetro é o **sep**, o separador de colunas do arquivo. O Pandas espera vírgula por padrão — é assim que ele sabe onde termina uma coluna e começa a próxima.

O problema: uma planilha exportada do Excel em português costuma usar **ponto e vírgula** como separador. Se você não avisar o Pandas, ele tenta separar pela vírgula e as colunas vêm todas coladas numa só — exatamente o que aconteceu ao lado, ao passar **sep=";"** numa base que na verdade usa vírgula.

A lição prática: sempre confira o resultado depois de carregar. Se as colunas vierem todas grudadas num campo só, é sinal de que o **sep** está errado.

low_memory=False: o Aviso de Colunas Mistas

Terceiro parâmetro: **low_memory=False**. Ele resolve um aviso comum — não é erro, é só aviso — de que uma coluna tem tipos de dados misturados, como datas e textos juntos na mesma coluna.

Por padrão, o Pandas lê a base em pedaços para economizar memória, e por isso pode não identificar corretamente o tipo de cada coluna de uma vez.

Ao lado: passando **low_memory=False**, você diz para ele carregar a base inteira de uma só vez — o aviso desaparece. O custo é usar mais memória do computador para isso.



	ID_Pedido	Data	Loja	Produto	Preco_Unitario	Qtd	Cliente	Data_Base
0	1	2023-06-08	São Paulo	Mouse Gamer	120.0	1	Cliente_4095	2025-01-01
1	2	2023-03-01	Belo Horizonte	iPhone 14	5500.0	1	Cliente_8750	NaN
2	3	2023-02-25	NaN	Monitor 27"	1200.0	1	Cliente_14859	NaN
3	4	2024-11-19	RIO DE JANEIRO	Mouse Gamer	120.0	2	Cliente_17343	NaN
4	5	2024-01-27	Rio de Janeiro	Smartphone Samsung	2200.0	1	Cliente_23377	NaN

O Que Procurar: Problemas Comuns na Base

Com a base carregada, começa a etapa de **inspeção**: olhar com atenção para encontrar problemas antes de sair analisando os dados. E normalmente vai ter — esse é um exemplo real, direto da nossa base.

Ao lado, dois problemas clássicos: a linha 2 tem a coluna **Loja** vazia — **NaN** ("not a number"), o marcador que o Pandas usa para qualquer valor ausente, não só números. E as linhas 3 e 4 têm a mesma cidade escrita de duas formas diferentes: **RIO DE JANEIRO** e **Rio de Janeiro**.

Para o Pandas, essas duas strings são valores completamente diferentes — o que vai bagunçar qualquer contagem por loja mais adiante. Encontrar esse tipo de problema é o objetivo da inspeção.

	ID_Pedido	Data	Loja	Produto	Preco_Unitario	Qtd	Cliente	Data_Base
100085	90626	2023-01-25	Porto Alegre	iPhone 14	5500.0	1	Cliente_14913	NaN
100086	18337	2023-10-04	Curitiba	Notebook Dell	3500.0	2	Cliente_16605	NaN
100087	81502	2024-06-14	São Paulo	Notebook HP	3200.0	1	Cliente_19663	NaN
100088	92089	2024-06-12	Porto Alegre	Teclado Mecânico	250.0	1	Cliente_21537	NaN
100089	169	2023-10-10	Recife	Mouse Gamer	120.0	3	Cliente_4407	NaN

head() e tail(): as Pontas da Base

Por padrão, **display(df_vendas)** mostra só as 5 primeiras e as 5 últimas linhas — pouco para inspecionar uma base desse tamanho.

Para ver mais, existe o **head()**, que traz o início da base, e o **tail()**, o final dela. Os dois aceitam a quantidade de linhas como argumento: **df_vendas.head(15)** ou **df_vendas.tail(15)**.

Ao lado, o resultado de **tail(15)**: as 15 últimas linhas da base, terminando exatamente nas mesmas 5 linhas finais que já apareciam no display padrão — só que agora com bem mais contexto ao redor.

sample(): uma Amostra Aleatória da Base

Diferente do **head()** e do **tail()**, que sempre mostram as pontas da base, o **sample()** pega linhas **aleatórias** — nem as primeiras, nem as últimas.

A sintaxe é a mesma: **df_vendas.sample(15)** traz 15 linhas escolhidas ao acaso. É útil quando você quer uma ideia geral da base sem viés de posição.

Ao lado, o resultado: repare nos índices — 43925, 96636, 29556... — completamente fora de ordem, confirmando que são linhas aleatórias de qualquer ponto da base, não um pedaço contínuo.

	ID_Pedido	Data	Loja	Produto	Preco_Unitario	Qtd	Cliente	Data_Base
43925	43926	2023-02-10	Curitiba	Notebook HP	3200.0	1	Cliente_17592	NaN
96636	96637	2023-06-22	Belo Horizonte	Monitor 27"	1200.0	1	Cliente_5741	NaN
29556	29557	2023-09-10	Recife	Smartphone Samsung	2200.0	1	Cliente_17504	NaN
3859	3860	2024-07-19	São Paulo	Monitor 27"	1200.0	1	Cliente_1158	NaN
57623	57624	2024-09-02	Belo Horizonte	iPhone 14	5500.0	1	Cliente_27547	NaN
34444	34445	2024-11-09	Porto Alegre	Mouse Gamer	120.0	2	Cliente_19955	NaN
69026	69027	2024-12-10	Salvador	Notebook HP	3200.0	1	Cliente_2845	NaN
82896	82897	2024-05-25	Porto Alegre	Notebook Dell	3500.0	5	Cliente_14645	NaN
24728	24729	2024-01-22	RIO DE JANEIRO	Cabo HDMI	40.0	1	Cliente_10288	NaN
72938	72939	2023-10-05	Porto Alegre	Mouse Gamer	120.0	1	Cliente_10335	NaN
80153	80154	2023-09-25	Belo Horizonte	Notebook Dell	3500.0	1	Cliente_21729	NaN

```
(100100, 8)

['ID_Pedido',
 'Data',
 'Loja',
 'Produto',
 'Preco_Unitario',
 'Qtd',
 'Cliente',
 'Data_Base']

<class 'pandas.core.frame.DataFrame'>
```

shape, columns e Índice: a Estrutura da Base

Toda base do Pandas é organizada em colunas (com um rótulo, o nome) e linhas — e as linhas também têm um rótulo, que o Pandas chama de **índice**. Quando você carrega uma base do zero, o índice começa em 0 e vai até a última linha — um **RangeIndex**.

Se você só quer saber o tamanho da base sem dar um display completo, use **df_vendas.shape** — repare que não tem parênteses, porque **shape** não é uma função, é uma característica da base. Ao lado: **(100100, 8)** — 100.100 linhas e 8 colunas.

Já **list(df_vendas.columns)** devolve todos os nomes de coluna como uma lista Python comum — dá para tratar, percorrer ou filtrar essa lista como qualquer outra.

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100100 entries, 0 to 100099
Data columns (total 8 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   ID_Pedido       100100 non-null  int64
 1   Data            100100 non-null  object
 2   Loja            98099 non-null   object
 3   Produto         100100 non-null  object
 4   Preco_Unitario  100100 non-null  float64
 5   Qtd             100100 non-null  int64
 6   Cliente         100100 non-null  object
 7   Data_Base       1 non-null       object
dtypes: float64(1), int64(2), object(5)
memory usage: 6.1+ MB

None
```

	ID_Pedido	Preco_Unitario	Qtd
count	100100.000000	100100.000000	100100.000000

info(): a Estrutura da Base num Golpe Só

Depois de explorar linhas soltas com **head()**, **tail()** e **sample()**, chega a hora de ver a base inteira de uma vez. **display(df_vendas.info())** traz um resumo técnico completo.

Ele mostra, para cada coluna, o **Non-Null Count** (quantos valores estão preenchidos) e o **Dtype** (o tipo de dado). Comparando com o total de linhas (100.100), já dá para ver problemas: a coluna **Loja** tem só 98.099 preenchidas, e a **Data** aparece como **object** (texto), não como data de verdade.

E a coluna **Data_Base** tem só **1 non-null** de 100.100 linhas — praticamente vazia, candidata a ser descartada. No rodapé, um resumo geral de quantas colunas de cada tipo a base tem e quanta memória ela ocupa.



	ID_Pedido	Preco_Unitario	Qtd
count	100100.000000	100100.000000	100100.000000
mean	50004.810180	2000.204595	1.499101
std	28866.872543	1841.050733	1.241605
min	1.000000	40.000000	1.000000
25%	25008.750000	120.000000	1.000000
50%	50004.500000	1200.000000	1.000000
75%	75002.250000	3200.000000	1.000000
max	100000.000000	5500.000000	10.000000

describe(): Resumo Estatístico das Colunas Numéricas

Enquanto **info()** fala de estrutura, **display(df_vendas.describe())** fala de números: um resumo estatístico automático de toda coluna numérica (**int64** ou **float64**) da base — aqui, **ID_Pedido**, **Preco_Unitario** e **Qtd**.

Para cada uma: **count** (quantos valores), **mean** (a média), **std** (o desvio padrão) e os três quartis — **25%**, **50%** (a mediana) e **75%**.

Sem entrar na estatística por trás, o describe já ajuda a enxergar a faixa de valores da base — ao lado, o **Preco_Unitario** varia de 40 a 5.500, com metade dos pedidos abaixo de 1.200. Com base carregada e inspecionada, começa a etapa de tratamento.

Selecionando Colunas: Colchete Único e Duplo

Com a base inspecionada, o primeiro passo do tratamento é entender como selecionar colunas. Para uma única coluna, você usa a base seguida do nome dela entre colchetes: df_vendas["Loja"]. O nome precisa ser **exatamente igual** ao que aparece na base.

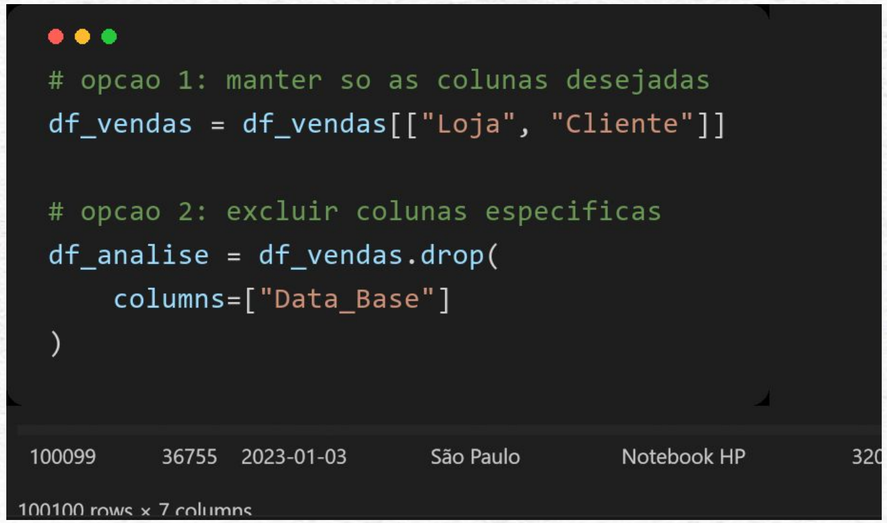
Repare no resultado ao lado: uma única coluna vem sem formatação de tabela, só uma lista de valores com o índice do lado — porque isso é uma **Series**, não um DataFrame. Ela funciona, por trás dos panos, como um array do NumPy — dá para operar colunas inteiras de uma vez, sem precisar de um **for**.

Para selecionar mais de uma coluna, um colchete não basta — você precisa de **dois**: df_vendas[["Loja", "Cliente"]]. O colchete de fora diz "selecione colunas", o de dentro é porque você está passando uma lista de nomes. Esse resultado já vem formatado como tabela, porque é um DataFrame de verdade.

```
# tratamento de dados
display(df_vendas["Loja"])
display(df_vendas[["Loja", "Cliente"]])
```

```
0              São Paulo
1         Belo Horizonte
2                    NaN
3         RIO DE JANEIRO
4         Rio de Janeiro
               ...
100095      Porto Alegre
100096      Porto Alegre
100097    Belo Horizonte
100098         Salvador
100099        São Paulo
Name: Loja, Length: 100100, dtype: object
```

	Loja	Cliente
0	São Paulo	Cliente_4095
1	Belo Horizonte	Cliente_8750
2	NaN	Cliente_14859
3	RIO DE JANEIRO	Cliente_17343
4	Rio de Janeiro	Cliente_23377
...	...	...
100095	Porto Alegre	Cliente_11755
100096	Porto Alegre	Cliente_12879



Excluindo Colunas e Trabalhando numa Cópia

A base tem uma coluna praticamente inútil, a **Data_Base** (só 1 valor preenchido em 100.100). Existem duas formas de excluir colunas: selecionar só as que você quer manter, ou usar **.drop(columns=[...])** passando as que você quer descartar — mais direto quando são poucas colunas.

Repare que reescrever **df_vendas** por cima dela mesma edita a base original — se você rodar aquela célula de novo, ela já não vai ter mais a coluna removida. Por isso a prática mais segura é criar uma **cópia** para fazer todos os tratamentos: **df_analise**. A base original fica intacta, e você sempre pode voltar a ela se precisar recomeçar do zero.

Depois de excluir a coluna, a base cai de 8 para 7 colunas — confirmado no resultado real: **100100 rows × 7 columns**. A única exceção à regra da cópia é quando a base é tão grande que já pesa na memória do computador.

```
# nulos
# df_analise = df_analise.dropna() # excluir todas as linhas que têm 1 va
df_analise["Loja"] = df_analise["Loja"].fillna("Online")
```

	ID_Pedido	Data	Loja	Produto	Preco_Unita
0	1	2023-06-08	São Paulo	Mouse Gamer	12
1	2	2023-03-01	Belo Horizonte	iPhone 14	550
2	3	2023-02-25	Online	Monitor 27"	120
3	4	2024-11-19	RIO DE JANEIRO	Mouse Gamer	12
4	5	2024-01-27	Rio de Janeiro	Smartphone Samsung	220

fillna(): Preenchendo em vez de Excluir

Para saber quais colunas têm valores vazios, basta olhar o **info()** — como a **Data_Base** já foi excluída, sobrou só a coluna **Loja** com valores vazios (98.099 de 100.100 preenchidas).

A forma mais simples é o **dropna()**, que remove toda linha com pelo menos um valor vazio — deixado comentado ao lado como alternativa. Mas excluir 2 mil linhas só porque a Loja estava vazia pode jogar fora informação boa.

Se você sabe **por que** está vazia — aqui, regra de negócio: loja vazia é venda **online** — dá para preencher em vez de excluir: **df_analise["Loja"].fillna("Online")**. Ao lado, o resultado real: a linha 2, antes vazia, agora mostra **Online**.

Convertendo para Data: pd.to_datetime()

Vazios tratados, próximo problema que o **info()** revelou: a coluna **Data** está com **Dtype object** — o Pandas está lendo as datas como texto puro, não como data de verdade.

Para converter, existe **pd.to_datetime()**, que transforma a coluna para o formato **DateTime** — o padrão de datas do Python. Você também precisa dizer a ele qual é o formato da data (ano-mês-dia, dia/mês/ano etc.) através do parâmetro **format**.

Ao lado: **format="%Y-%m-%d"** (ano com 4 dígitos, mês, dia). Depois de rodar, o **info()** confirma: a coluna **Data** agora é **datetime64[ns]**, não mais **object**.

```
df_analise["Data"] = pd.to_datetime(df_analise["Data"], format="%Y-%m-%d")
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100100 entries, 0 to 100099
Data columns (total 7 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   ID_Pedido       100100 non-null  int64
 1   Data            100100 non-null  datetime64[ns]
 2   Loja            100100 non-null  object
 3   Produto         100100 non-null  object
```

		Sonntag, Montag, ..., Samstag (de_DE)	
%w	Weekday as a decimal number, where 0 is Sunday and 6 is Saturday.	0, 1, ..., 6	
%d	Day of the month as a zero-padded decimal number.	01, 02, ..., 31	(9)
%b	Month as locale's abbreviated name.	Jan, Feb, ..., Dec (en_US); Jan, Feb, ..., Dez (de_DE)	(1)
%B	Month as locale's full name.	January, February, ..., December (en_US); Januar, Februar, ..., Dezember (de_DE)	(1)
%m	Month as a zero-padded decimal number.	01, 02, ..., 12	(9)
%y	Year without century as a zero-padded decimal number.	00, 01, ..., 99	(9)
%Y	Year with century as a decimal number.	0001, 0002, ..., 2013, 2014, ..., 9998, 9999	(2)

O Parâmetro format: de Onde Vêm os Códigos

Você não precisa decorar os códigos de formato de data — a própria documentação oficial do Python (procure "strftime and strptime behavior") tem a tabela completa, com hora, minuto, segundo e dia da semana inclusos.

Ao lado, um pedaço real dessa tabela: **%d** é o dia com 2 dígitos, **%m** o mês com 2 dígitos, **%Y** o ano com 4 dígitos. É por isso que **"2024-01-27"** (ano-mês-dia) vira **format="%Y-%m-%d"**.

Repare que maiúsculas e minúsculas fazem diferença: **%m** (mês) é diferente de **%M** (minuto) — um erro comum de quem está começando com formatação de datas.

```
# padronizacao
df_analise["Loja"] = df_analise["Loja"].str.strip()
df_analise["Loja"] = df_analise["Loja"].str.title()
```

	ID_Pedido	Data	Loja	Produto	Preco_Unitar
0	1	2023-06-08	São Paulo	Mouse Gamer	120
1	2	2023-03-01	Belo Horizonte	iPhone 14	5500
2	3	2023-02-25	Online	Monitor 27"	1200
3	4	2024-11-19	Rio De Janeiro	Mouse Gamer	120
4	5	2024-01-27	Rio De Janeiro	Smartphone Samsung	2200
...	...	...	...	...	
100095	94091	2023-10-20	Porto Alegre	iPhone 14	5500
100096	52883	2024-03-17	Porto Alegre	Notebook Dell	3500
100097	65070	2023-06-19	Belo Horizonte	Smartphone Samsung	2200
100098	94031	2024-06-14	Salvador	iPhone 14	5500
100099	36755	2023-01-03	São Paulo	Notebook HP	3200

Padronizando o Texto: strip() e title()

Um problema que só aparece quando você já está analisando: "RIO DE JANEIRO" e "Rio de Janeiro" são a mesma cidade para uma pessoa, mas para o Pandas são **dois valores diferentes** — o que separa as vendas da mesma loja em linhas distintas do relatório.

Para trabalhar com texto de forma eficiente (sem for), usa-se **.str**, que disponibiliza as funções de texto do Python direto na coluna. Aqui, duas bastam: **strip()** remove espaços em branco no início e no fim, e **title()** deixa a primeira letra de cada palavra maiúscula e o resto minúsculo.

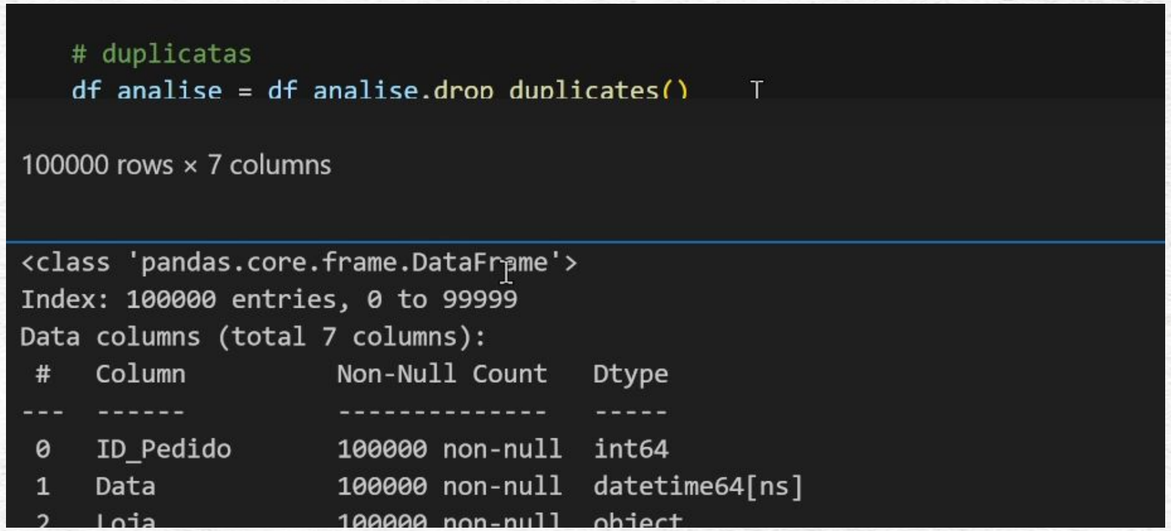
Ao lado, o resultado real: as linhas 3 e 4, que antes tinham **RIO DE JANEIRO** e **Rio de Janeiro** escritos de formas diferentes, agora aparecem as duas como **Rio De Janeiro** — o Pandas já reconhece como o mesmo valor.

drop_duplicates(): Removendo Linhas Idênticas

Último tratamento: linhas **completamente duplicadas** — dois pedidos com todas as informações iguais, o que não faz sentido numa tabela de pedidos e costuma indicar um erro no preenchimento.

O método **drop_duplicates()** resolve isso direto: **df_analise = df_analise.drop_duplicates()**. Ele também aceita dois parâmetros úteis: **subset** (uma lista de colunas que define o que conta como duplicata, em vez da linha inteira) e **keep** (qual cópia manter — "first" por padrão, ou "last").

Ao lado, o resultado real: a base cai de **100.100** para **100.000** linhas — as 100 duplicatas foram removidas. Rodando o **info()** de novo, tudo confirma certinho. Assim termina a etapa de tratamento de dados desta primeira parte do projeto.



```
# duplicatas
df_analise = df_analise.drop_duplicates()
```

100000 rows × 7 columns

```
<class 'pandas.core.frame.DataFrame'>
Index: 100000 entries, 0 to 99999
Data columns (total 7 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   ID_Pedido       100000 non-null  int64
 1   Data            100000 non-null  datetime64[ns]
 2   Loja            100000 non-null  object
```

```
# criar novas colunas

# faturamento
df_analise["Faturamento"] = df_analise["Qtd"] * df_analise["Preco_Unitario"]

# regiao

# forma de venda

display(df_analise)
```

✓ 0.0s

	ID_Pedido	Data	Loja	Produto	Preco_Unitario	Qtd	Cliente	Faturamento
0	1	2023-06-08	São Paulo	Mouse Gamer	120.0	1	Cliente_4095	120.0
1	2	2023-03-01	Belo Horizonte	iPhone 14	5500.0	1	Cliente_8750	5500.0
2	3	2023-02-25	Online	Monitor 27"	1200.0	1	Cliente_14859	1200.0
3	4	2024-11-19	Rio De Janeiro	Mouse Gamer	120.0	2	Cliente_17343	240.0
4	5	2024-01-27	Rio De Janeiro	Smartphone Samsung	2200.0	1	Cliente_23377	2200.0
...	...	...	...	...	...	...	...	...
99995	99996	2023-01-24	São Paulo	Mouse Gamer	120.0	2	Cliente_13732	240.0
99996	99997	2024-08-28	Rio De Janeiro	Cabo HDMI	40.0	1	Cliente_25058	40.0

Engenharia de Colunas: Faturamento

Com o tratamento concluído, começa a etapa de engenharia de colunas — criar colunas novas com base nas que já existem. Como saber quais colunas criar? Você parte das análises que quer fazer: aqui, o CEO pediu faturamento por gerente, então precisamos de uma coluna de faturamento.

A primeira e mais direta forma de criar uma coluna é uma operação entre colunas existentes: **df_analise["Faturamento"] = df_analise["Qtd"] * df_analise["Preco_Unitario"]**. É a grande vantagem do Pandas — em vez de um for linha a linha, você multiplica a coluna inteira de uma vez.

Repare a importância do tratamento feito antes: se **Qtd** tivesse algum valor vazio, o faturamento daquela linha também sairia vazio. Tudo que você constrói a partir daqui depende da base já estar tratada.

np.where(): Criando a Coluna Forma_de_Venda

A segunda forma de criar colunas é aplicar uma condição. Aqui, o objetivo é uma coluna que diga se a venda foi **online** ou **presencial**.

Para colunas condicionais, o recomendado é o **np.where()** — funciona como um **if** aplicado direto na coluna inteira: condição, valor se verdadeira, valor se falsa. É como o **WHERE** do SQL, para quem já conhece.

Ao lado: **np.where(df_analise["Loja"]=="Online", "Online", "Presencial")**. Se a loja for igual a "Online", preenche "Online"; caso contrário, "Presencial". Dá para encadear vários **np.where()** um dentro do outro se precisar de mais de duas opções.

```
# forma de venda
df_analise["Forma_de_Venda"] = np.where(df_analise["Loja"]=="Online", "Online", "Presencial")

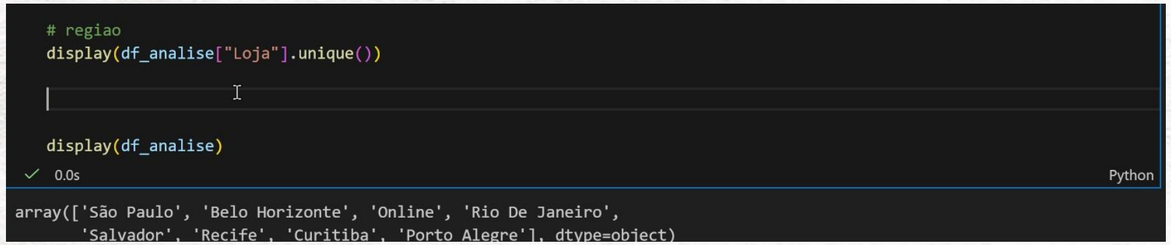
# regiao
df_analise["Região"] =



display(df_analise)
```

0.0s Python

	ID_Pedido	Data	Loja	Produto	Preco_Unitario	Qtd	Cliente	Faturamento	Forma_de_Venda
0	1	2023-06-08	São Paulo	Mouse Gamer	120.0	1	Cliente_4095	120.0	Presencial
1	2	2023-03-01	Belo Horizonte	iPhone 14	5500.0	1	Cliente_8750	5500.0	Presencial



Descobrindo os Valores Únicos: .unique()

Última coluna nova: **Região** — sudeste, nordeste, sul ou online. Antes de criar essa coluna, é preciso saber quais cidades existem na base.

Para isso existe o **.unique()**: aplicado numa coluna, devolve todos os valores diferentes que aparecem nela, sem repetição — ao lado, **df_analise["Loja"].unique()** revela as 8 cidades da base.

Esse é o padrão geral do Pandas para operações numa coluna específica: nome da coluna, ponto, nome da operação. **.sum()** soma, **.count()** conta valores, **.value_counts()** conta quantas vezes cada valor aparece.

```
    'Rio De Janeiro': "Sudeste",
    'Salvador': "Nordeste",
    'Recife': "Nordeste",
    'Curitiba': "Sul",
    'Porto Alegre': "Sul"
}

df_analise["Região"] = df_analise["Loja"].map(dic_regioes)
```

	ID_Pedido	Data	Loja	Produto	Preco_Unitario	Qtd	Cliente	Faturamento	Forma_de_Venda	Região
0	1	2023-06-08	São Paulo	Mouse Gamer	120.0	1	Cliente_4095	120.0	Presencial	Sudeste
1	2	2023-03-01	Belo Horizonte	iPhone 14	5500.0	1	Cliente_8750	5500.0	Presencial	Sudeste
2	3	2023-02-25	Online	Monitor 27"	1200.0	1	Cliente_14859	1200.0	Online	Online

Mapeamento com Dicionário e .map()

Com as 8 cidades em mãos, a terceira forma de criar colunas é o **mapeamento**: para cada valor de entrada existe um valor de saída correspondente — exatamente o que um **dicionário** faz.

O **.map()** aceita duas formas de uso. Você pode passar uma **função** — útil quando a lógica é mais complexa, tipo um condicional dentro dela — que recebe o valor da coluna e devolve o que deve ser preenchido.

Mas para um mapeamento simples como este, dá para passar o dicionário direto: **df_analise["Região"] = df_analise["Loja"].map(dic_regioes)** — para cada loja, ele busca a chave correspondente e preenche o valor. Mais rápido de escrever quando não precisa de lógica extra.

isna().sum(): Conferindo se o Mapeamento Funcionou

Com as três colunas novas criadas (**Faturamento**, **Forma_de_Venda**, **Região**), vale conferir se o **.map()** realmente funcionou para todas as lojas — se alguma não bater com o dicionário, a região daquela linha fica vazia.

Dá para rodar o **info()** de novo, mas para ver só a quantidade de vazios existe algo mais direto: **isna()** (é vazio?) aplicado na base inteira, seguido de **sum()** para somar os vazios de cada coluna.

Ao lado, o início do resultado: zero em todas as colunas — confirmando que o mapeamento cobriu 100% das lojas. Esse é o mesmo padrão de aplicar uma função na base inteira (por coluna) em vez de numa coluna só.

```
...  ID_Pedido        0
     Data             0
     Loja             0
     Produto          0
     Preco_Unitario   0
```

```python
#.loc: id do pedido -> por nome da coluna e por nome da linha
id_pedido = 4

loja = df_analise.loc[3, "Loja"]
produto = df_analise.loc[3, "Produto"]
cliente = df_analise.loc[3, "Cliente"]
print(loja, produto, cliente)

#.iloc -> por posicao
id_pedido = 4
loja = df_analise.iloc[3, 2]
produto = df_analise.iloc[3, 3]
cliente = df_analise.iloc[3, 6]

print(loja, produto, cliente)
```

.loc e .iloc: Localizando por Nome ou por Posição

Com as colunas prontas, começa a etapa de análise — e a primeira habilidade é filtrar: pegar só o pedaço da base que interessa. Existem quatro grandes formas: selecionar colunas (já visto), **.loc**, **.iloc** e condicional — a condicional é disparada a mais usada.

O **.loc** localiza pelo **nome do índice** e pelo **nome da coluna**. Ao lado, para o pedido de índice 3: **df_analise.loc[3, "Loja"]** e o mesmo para Produto e Cliente.

O **.iloc** localiza sempre por **posição**, não pelo nome do índice. Com a base ainda na ordem original, índice e posição coincidem — **iloc[3, 2]** dá o mesmo resultado que **loc[3, "Loja"]**.

```
df_analise = df_analise.sort_values(by="Data")
```

```
Rio De Janeiro Mouse Gamer Cliente_17343
Salvador Mouse Gamer Cliente_28168
```

	ID_Pedido	Data	Loja	Produto	Preco_Unitario	Qtd	Cliente	Faturamento	Forma_de_Venda	Região
34749	34750	2023-01-01	Rio De Janeiro	Notebook HP	3200.0	1	Cliente_9014	3200.0	Presencial	Sudeste
56681	56682	2023-01-01	Porto Alegre	Notebook HP	3200.0	3	Cliente_25493	9600.0	Presencial	Sul
51841	51842	2023-01-01	Recife	Smartphone Samsung	2200.0	1	Cliente_14591	2200.0	Presencial	Nordeste
54076	54077	2023-01-01	Salvador	Mouse Gamer	120.0	1	Cliente_28168	120.0	Presencial	Nordeste

Ordenando com sort_values(): o .iloc Muda, o .loc Não

Para ordenar a base, existe o **sort_values()**, passando o parâmetro **by** com o nome da coluna de referência: **df_analise = df_analise.sort_values(by="Data")**.

O efeito colateral: os índices ficam fora de ordem, porque cada linha carrega o índice que tinha antes de ordenar. Isso é exatamente o que expõe a diferença entre **.loc** e **.iloc**.

Ao lado, o resultado real depois de ordenar: **.loc[3, "Loja"]** continua achando **Rio De Janeiro** (o índice 3 não mudou de dono). Mas **.iloc[3, 2]** agora devolve **Salvador** — porque a posição 3 da tabela mudou depois da ordenação.

reset_index(): Recomeçando a Contagem do Índice

Depois de ordenar, normalmente você quer que o índice volte a ir de 0 até a última linha, do jeito que é mais fácil de ler. Para isso existe o **reset_index()**.

Passando **drop=True**, o índice antigo (o embaralhado) é descartado — sem isso, ele viraria uma coluna nova na base, o que raramente é o que você quer.

Ao lado, o resultado: novo índice de 0 a 4 nas primeiras linhas, e agora **.loc** e **.iloc** voltam a coincidir — os dois encontram **Rio De Janeiro, Teclado Mecânico, Cliente_24330**. Boa prática: textos ficam em colunas, o índice é só um número.

```
df_analise = df_analise.reset_index(
    drop=True
)
```

```
Rio De Janeiro Teclado Mecânico Cliente_24330
Rio De Janeiro Teclado Mecânico Cliente_24330
```

	ID_Pedido	Data	Loja	Produto	Preco_Unitario	Qtd	Cliente	Faturamento	Forma_de_Venda	Região
		2023	Rio De	Notebook						

```
# df_id_pedido_4 = df_analise[df_analise["ID_Pedido"]==4]

# exportar pedaços da base
df_vendas_sp = df_analise[df_analise["Loja"]=="São Paulo"]
df_vendas_sp.to_csv("Vendas_SP.csv", index=False)

# exportar as vendas de 2024
df_vendas_2024 = df_analise[df_analise["Data"]>="2024-01-01"]
```

```
Rio De Janeiro
Mouse Gamer
Cliente_17343
Porto Alegre Cabo HDMI Cliente_22886
```

	ID_Pedido	Data	Loja	Produto	Preco_Unitario	Qtd	
50423	22735	2024-01-01	São Paulo	Cabo HDMI	40.0	1	Cliente
50424	31751	2024-01-01	Curitiba	Cabo HDMI	40.0	1	Cliente
50425	36481	2024-01-01	Salvador	Cabo HDMI	40.0	1	Client

Exportando Pedaços da Base: to_csv() e Filtro por Data

Imagine que o chefe pede uma base só com as vendas de São Paulo, para mandar para alguém. O filtro condicional resolve: **df_vendas_sp = df_analise[df_analise["Loja"]=="São Paulo"]**.

Para exportar, existe o **.to_csv()**: **df_vendas_sp.to_csv("Vendas_SP.csv", index=False)**. O **index=False** evita que a coluna de índice vire uma coluna a mais no arquivo — quase nunca é o que você quer.

Para filtrar por data, dá para comparar a coluna inteira como texto no formato ano-mês-dia: **df_analise["Data"]>="2024-01-01"** — mesmo resultado que usar **.dt.year==2024**, mas mais flexível para intervalos que não fecham num ano redondo.

Combinando Condições: & e |

Para filtrar por mais de uma condição ao mesmo tempo — por exemplo, só vendas de **Cabo HDMI** na região **Sul** — combine duas condições com **&** ("e", as duas precisam ser verdadeiras) ou **|** ("ou", uma ou outra, ou as duas).

Ao lado: **df_analise[(df_analise["Produto"]=="Cabo HDMI") & (df_analise["Região"]=="Sul")]** — repare que cada condição fica entre parênteses. Sem isso, o Pandas pode não entender onde uma condição termina e a outra começa, e a mensagem de erro costuma ser confusa de rastrear.

O resultado real confirma: só linhas de Cabo HDMI da região Sul (Porto Alegre, Curitiba) — nenhuma outra região ou produto passou pelo filtro.

```
# duplas condições
df_vendas_HDMI_SUL = df_analise[(df_analise["Produto"]=="Cabo HDMI") & (df_analise["Região"]=="Sul")]
```

	ID_Pedido	Data	Loja	Produto	Preco_Unitario	Qtd	Cliente	Faturamento	Forma_de_Venda	Região
0	1700	2023-01-01	Porto Alegre	Cabo HDMI	40.0	1	Cliente_14435	40.0	Presencial	Sul
2	35599	2023-01-01	Porto Alegre	Cabo HDMI	40.0	1	Cliente_18680	40.0	Presencial	Sul
3	37130	2023-01-01	Porto Alegre	Cabo HDMI	40.0	1	Cliente_22886	40.0	Presencial	Sul
4	49140	2023-01-01	Porto Alegre	Cabo HDMI	40.0	1	Cliente_24445	40.0	Presencial	Sul
		2023-01-	Porto	Cabo						

```
analise_lojas = df_analise[["Loja", "Faturamento"]].groupby("Loja").sum()
analise_lojas = analise_lojas.sort_values(by="Faturamento", ascending=Fal
analise_lojas = analise_lojas.reset_index()

display(analise_lojas)
```

	Loja	Faturamento
0	Salvador	R$42,300,610.00
1	Rio De Janeiro	R$42,294,720.00

groupby(): Ranking de Faturamento por Loja

Chegou a análise por agrupamento — qual loja vendeu mais? O **groupby()** agrupa uma coluna: cada valor aparece uma única vez no resultado.

Mas ele precisa saber o que fazer com as outras colunas — aplicar num DataFrame inteiro (sem filtrar) dá erro, porque ele tenta somar até colunas de texto e data. Por isso, seleciona-se só as colunas necessárias antes: **df_analise[["Loja", "Faturamento"]].groupby("Loja").sum()**.

Depois, **sort_values()** ordena por faturamento decrescente e **reset_index()** tira a loja de índice para virar coluna de novo. Ao lado, o resultado real já formatado com **.map("R${:,.2f}".format)**: Salvador lidera com R$42.300.610,00.

rename(): Renomeando Colunas para Apresentação

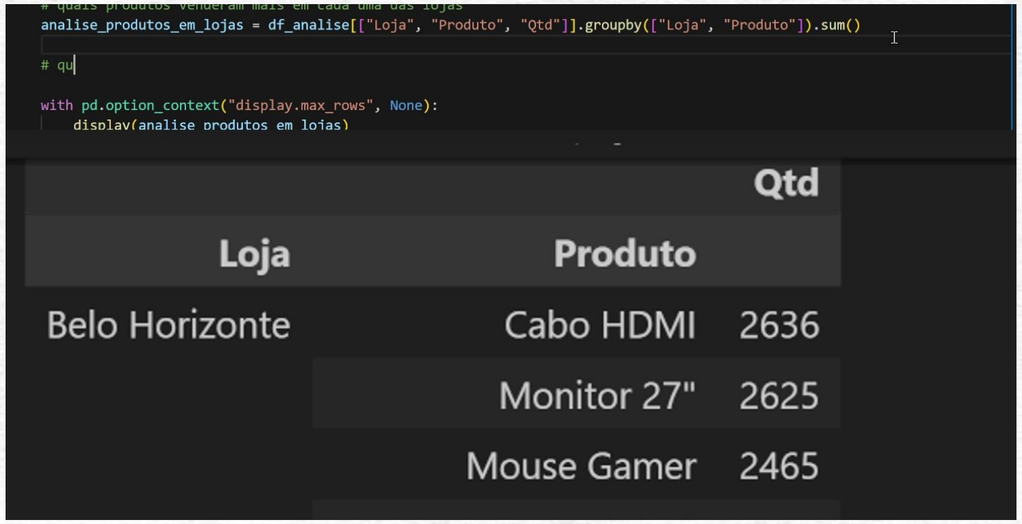
Nomes técnicos como **Qtd** funcionam no código, mas não numa apresentação para o chefe. Para trocar o nome de uma coluna existe o **rename()**.

A sintaxe usa um dicionário — a chave é o nome atual, o valor é o novo: **.rename(columns={"Qtd": "Vendas Totais"})**. Dá para renomear quantas colunas quiser de uma vez, todas no mesmo dicionário.

Ao lado, o resultado real do ranking de produtos mais vendidos no online: a coluna já aparece como **Vendas Totais**, não mais **Qtd**. Se você errar o nome da coluna atual, o Pandas simplesmente ignora — não dá erro, só não muda nada.

```
df_vendas_online = df_analise[df_analise["Loja"]=="Online"]
analise_produtos_online = df_vendas_online[["Produto", "Qtd"]].groupby("Produto").sum()
analise_produtos_online = analise_produtos_online.sort_values(by="Qtd", ascending=False)
analise_produtos_online = analise_produtos_online.rename(columns={"Qtd": "Vendas Totais"}) # alterar nome de coluna
```

Produto	Vendas Totais
Notebook HP	442
Cabo HDMI	403



groupby() com Múltiplas Colunas e option_context

E se a pergunta for mais específica — quais produtos mais venderam em cada loja? Isso exige agrupar por duas regras ao mesmo tempo, e a ordem delas importa: em vez de um nome só, passe uma **lista** para o **groupby()**: **groupby(["Loja", "Produto"])**.

Ele agrupa primeiro por loja e, dentro de cada loja, quebra por produto — o resultado vira um índice de dois níveis. Mas o display padrão do Pandas oculta linhas do meio quando a tabela é grande.

Para ver a tabela inteira só naquela linha (sem mudar o comportamento do resto do notebook), use **with pd.option_context("display.max_rows", None):** — tudo identado dentro do bloco mostra sem limite de linhas. Use com moderação: em tabelas muito grandes pode travar o editor.

Preparando o Cruzamento: Filtrar, Agrupar, Trazer a Meta

Próxima análise: quais gerentes bateram a meta em janeiro de 2023? Essa informação está espalhada em duas bases — as vendas (df_analise) e os gerentes de cada loja (df_gerentes, carregada lá no início, com Meta_Mensal por loja).

Primeiro, filtra só janeiro de 2023 combinando duas condições com &: (dt.year==2023) & (dt.month==1). Depois, agrupa por loja e soma o faturamento — com as_index=False, porque a coluna Loja vai servir para juntar com a outra base, e não pode virar índice.

Regra prática: quando a informação de fora só serve para uma correspondência final (não para o agrupamento em si), traga-a depois de agrupar.

```
# quais gerentes bateram a meta em janeiro de 2023
df_meta = df_analise[(df_analise["Data"].dt.year==2023) & (df_analise["Data"].dt.month==1)]

df_meta = df_meta[["Loja", "Faturamento"]].groupby("Loja", as_index=False).sum()
```

	A	B	C
1	Loja	Gerente	Meta_Mensal
2	São Paulo	Carlos	50,000

```
df_meta = df_meta.merge(df_gerentes, on="Loja", how="left")
```

	Loja	Faturamento	Gerente	Meta_Mensal
0	Belo Horizonte	1779100.0	Juliana	55000.0
1	Curitiba	1986920.0	Roberto	45000.0
2	Online	404570.0	NaN	NaN
3	Porto Alegre	1726640.0	Pedro	42000.0
4	Recife	1779020.0	Marcos	48000.0
5	Rio De Janeiro	1736830.0	NaN	NaN
6	Salvador	1686070.0	Ana	52000.0
7	São Paulo	1831140.0	Carlos	50000.0

Tipos de Join: por que how="left" é mais seguro

Para juntar duas bases usando uma correspondência de colunas, existe o **merge()** — passando **on="Loja"** quando o nome da coluna é igual nas duas tabelas. Por padrão, ele faz um **inner join**: só inclui no resultado as lojas que existem **nas duas** tabelas.

O problema: se uma loja não tiver gerente cadastrado, ela desaparece da tabela final sem aviso nenhum — perigoso, porque você pode nem perceber que uma loja sumiu.

Passando **how="left"**, o resultado garante todas as linhas da tabela da esquerda (**df_meta**), preenchendo com **NaN** onde não achar correspondência. Ao lado, o resultado real: **Online** (sem gerente mesmo) e **Rio De Janeiro** (só um problema de nome despadronizado entre as duas bases) aparecem com NaN — o que denuncia exatamente onde investigar.

Corrigindo a Correspondência e a Coluna Bateu Meta

O Rio De Janeiro sem gerente era um bug de padronização: a base de gerentes tinha o nome da loja escrito diferente da base de vendas. A correção: aplicar o mesmo .str.strip().str.title() que já era usado em df_analise também na base df_gerentes — e rodar tudo de novo (Run All).

Esse é o padrão de qualquer análise: você antecipa o máximo de tratamento possível, mas alguns problemas só aparecem quando o cruzamento de bases já está pronto.

Com as bases corrigidas, dá para responder a pergunta original: quais gerentes bateram a meta? Basta um np.where() comparando Faturamento com Meta_Mensal — ao lado, o resultado real: todo mundo bateu, menos o Online (que nem tem meta definida).

```
df_meta["Bateu Meta"] = np.where(df_meta["Faturamento"] >= df_meta["Meta_Mensal"], "Sim", "Não"
display(df_meta)
```

	ID_Pedido	Data	Loja	Produto	Preco_Unitario	Qtd	
0	1700	2023-01-01	Porto Alegre	Cabo HDMI	40.0	1	Client
1	12848	2023-01-01	Belo Horizonte	Cabo HDMI	40.0	1	Clier
2	35599	2023-01-01	Porto Alegre	Cabo HDMI	40.0	1	Client
3	37130	2023-01-01	Porto Alegre	Cabo HDMI	40.0	1	Client
4	49140	2023-01-01	Porto Alegre	Cabo HDMI	40.0	1	Client

	Loja	Faturamento	Gerente	Meta_Mensal	Bateu Meta
0	Belo Horizonte	1779100.0	Juliana	55000.0	Sim
1	Curitiba	1986920.0	Roberto	45000.0	Sim



Série Temporal: to_period() e .plot()

Última análise: como o faturamento evolui **mês a mês**, ao longo do tempo? Diferente das anteriores, essa é sensível à data — não basta agrupar uma característica, é preciso pensar em tendência.

Como a coluna **Data** já é do tipo data, dá para criar uma coluna de período com **df_analise["Data"].dt.to_period("M")** — agrupa automaticamente por mês e ano (**"2023-01"**, **"2023-02"**...). Depois, é só agrupar por essa coluna e somar o faturamento, igual sempre.

O Pandas já vem com gráficos do Matplotlib embutidos: **.plot()** numa tabela com índice de data gera o gráfico de linha automaticamente — como ao lado, oscilando entre 11,6 e 13 milhões, sem tendência clara de alta ou queda. Para gráficos mais bonitos, vale migrar para o Plotly depois. Com isso, encerra-se este exercício de revisão de Pandas.